



IBM App Connect Enterprise

ACE Toolkit and IBM Bob

1. INTRODUCTION.....	3
2. INSTALLING THE IBM BOB SHELL	4
3. CONFIGURING THE IBM BOB SHELL IN THE ACE TOOLKIT	6
4. REGISTER FOR THE IBM BOB TRIAL	12
5. DOWNLOAD AND CONFIGURE THE ACE-BOB IBM BOB SKILL.....	15
6. USE IBM BOB TO CREATE A SIMPLE HTTP FLOW	16
7. USE IBM BOB TO SUMMARIZE AN EXISTING FLOW	28
8. USE IBM BOB TO CREATE A SIMPLE FILE FLOW AND THEN TEST	33
END OF LAB GUIDE	43

1. Introduction

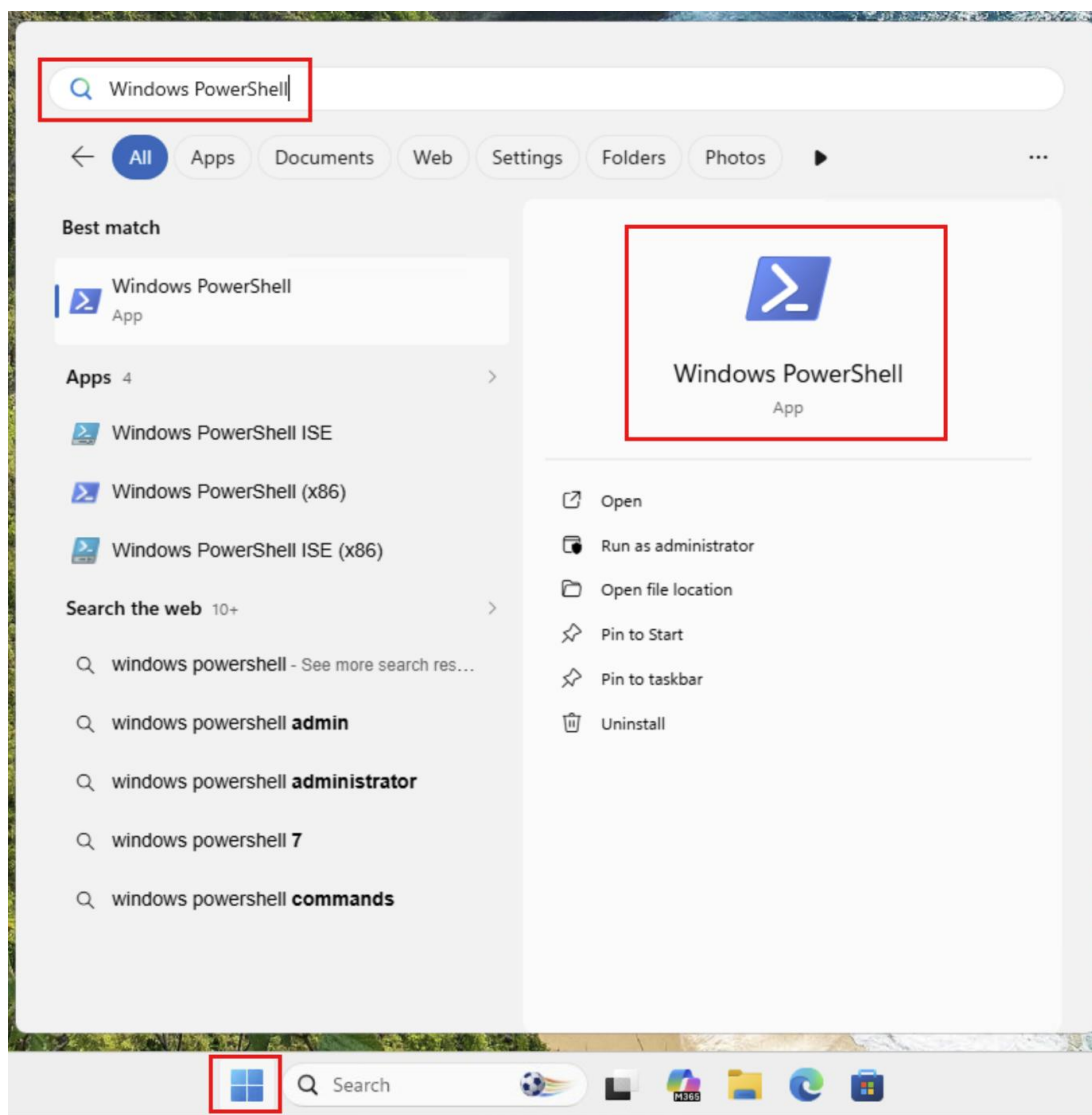
The purpose of this LAB is to showcase how to use IBM Bob Shell within the IBM App Connect Enterprise Toolkit. IBM Bob is the name of a recently released AI-Powered Development Partner and agentic tool which is designed to assist IT professionals throughout the entire Software Development Life Cycle. In particular this Lab demonstrates how IBM Bob can be used to help speed up development and flow authoring for ACE. IBM Bob provides a range of skills including the generation of new code, and the refactoring, analysis and summarization of existing code. Bob can read and write files directly, run terminal or shell commands and invoke Tools on MCP (Model Context Protocol) servers. IBM Bob can adapt to your specific needs with purpose-built modes that optimize desired behaviours for different situations. IBM Bob is often demonstrated as a plugin running inside Visual Studio Code, but this lab explores a terminal-based interface, called the IBM Bob Shell, which provides AI-assistance within terminal-based workflows. This user interface option slots nicely into an App Connect Enterprise Toolkit environment. The Lab will teach you how you can get the IBM Bob Shell running inside the ACE Toolkit, and how to configure Bob to use a specialized ace-bob skill that has been made publicly available in a repository on GitHub. Skills are reusable instruction sets that teach Bob new workflows and specialized tasks. Skills are like recipes that Bob can follow to complete specific types of work in a consistent, repeatable manner.

This lab assumes that you have at your disposal:

- A Windows system which has ACE 13 installed (the lab was written and tested using ACE 13.0.7.0). If you are attempting this lab as part of the ACE Early Experience Program workshop in Hursley, then you will have at your disposal a virtual environment which you can use if you don't have a local ACE installation already on your laptop.
- Authentication details and entitlement to use IBM Bob on the public cloud. In addition to the IBM Bob front end, Bob requires a back-end cloud hosted service which is separately charged and licensed to ACE itself. Charging for IBM Bob is done using consumption based plans. Each action you complete with Bob, such as generating code, analyzing files, or running commands, consumes a number of Bobcoins based on the computational resources required. Bobcoins are essentially resource units that provide a transparent, predictable way to measure and bill for your Bob usage. There are various available tiered plans which provide you a monthly allocation of Bobcoins, including a free trial. **The lab includes instructions for how to sign up to the IBM Bob free trial, which you will need to do using your IBMid.**

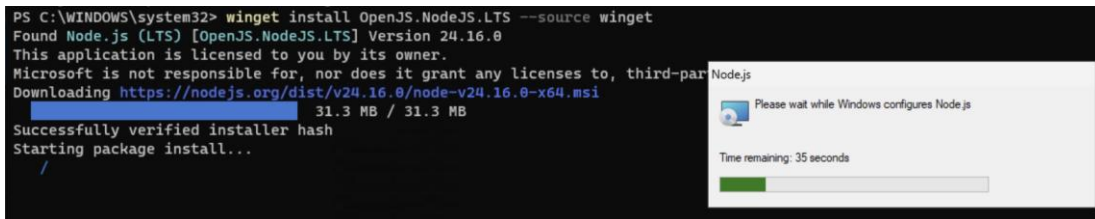
2. Installing the IBM Bob Shell

1. Log in to your Windows environment. If you are using the IBM provided environment, you can log in with userid **itzuser** and password **IBMDem0s**
2. From the Windows start menu, launch Windows Power Shell:

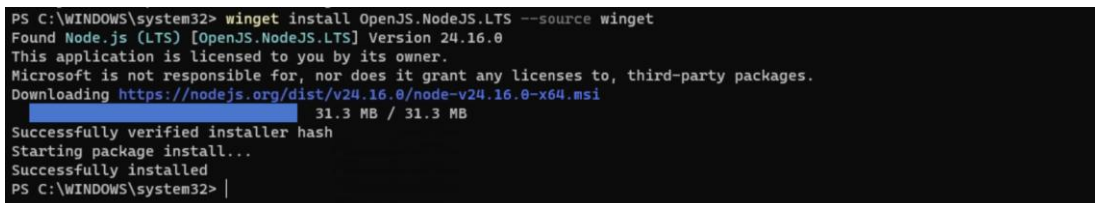


3. In the Windows PowerShell console, execute the following command to install nodeJS:

```
winget install OpenJS.NodeJS.LTS --source winget
```



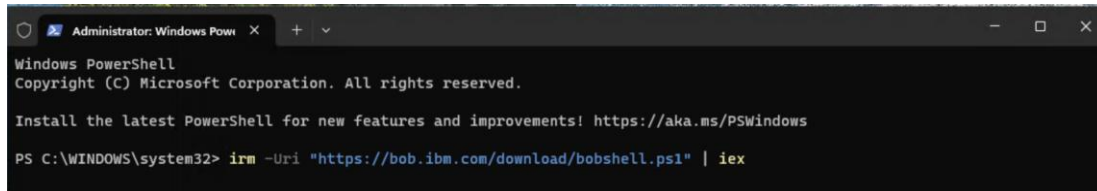
After a minute or two the install should complete successfully as shown:



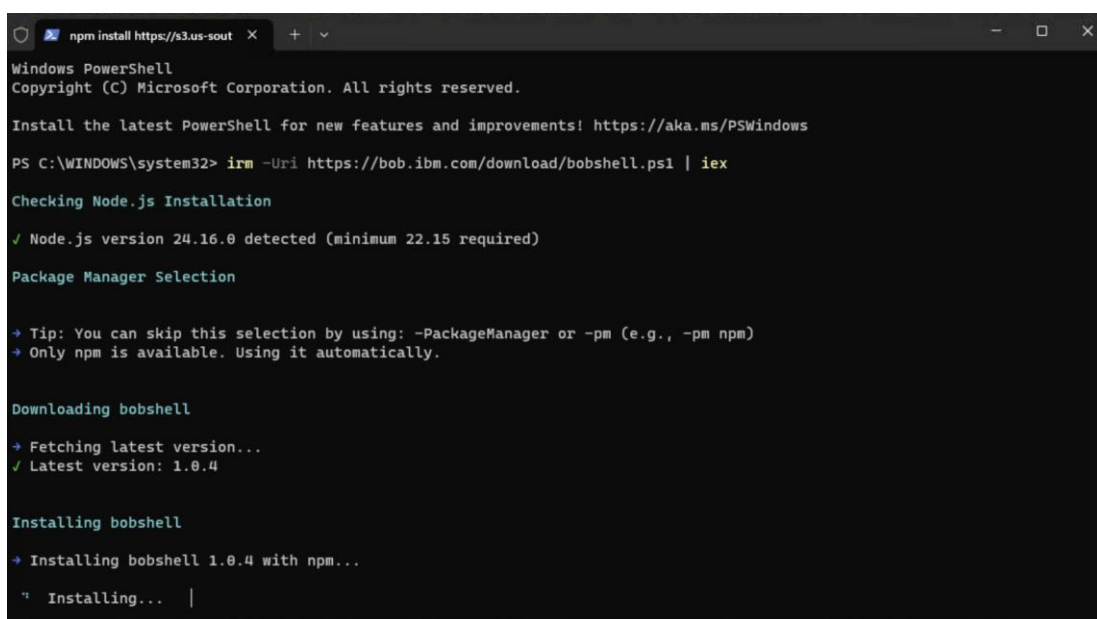
4. Close the Windows PowerShell console (to pick up the nodeJS installation you will need a new Windows PowerShell console to be opened which we will do in the next instruction step).

5. From the Windows start menu open a new Windows PowerShell console (as shown in the first step of this lab guide) and execute the following command to install IBM Bob Shell:

```
irm -Uri "https://bob.ibm.com/download/bobshell.ps1" | iex
```



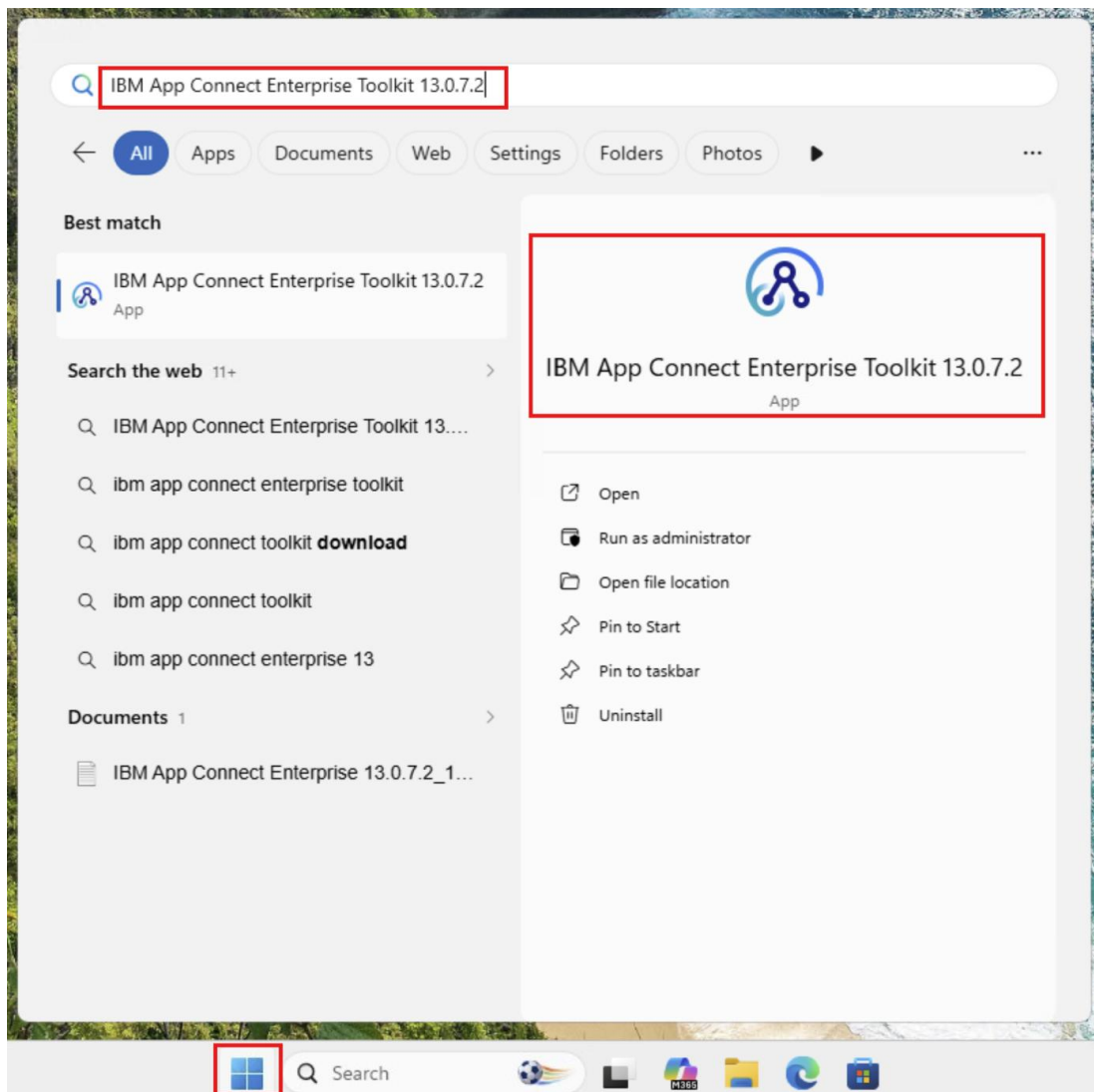
The install process will look like this:



When the install completes, you should find that IBM Bob Shell has been installed in the directory `C:\Users\<YourUser>\AppData\Roaming\npm` so if you are using the IBM provided virtual environment you will be using `userid` so the directory where IBM Bob Shell has been installed will be `C:\Users\ituser\AppData\Roaming\npm`

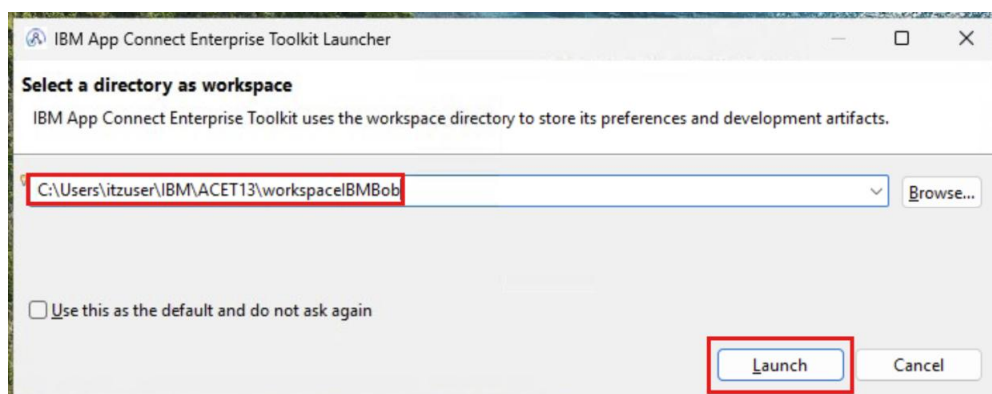
3. Configuring the IBM Bob Shell in the ACE Toolkit

6. In the Windows environment, from the Windows Start menu open the IBM App Connect Enterprise Toolkit 13.0.7.2:

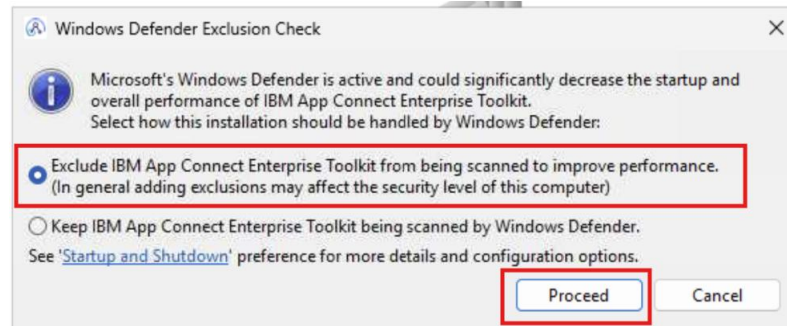


7. After the splash screen, the Toolkit will ask which workspace you wish to use. We will create a new workspace for this lab. In the IBM provided environment specify the following workspace location and then click the Launch button:

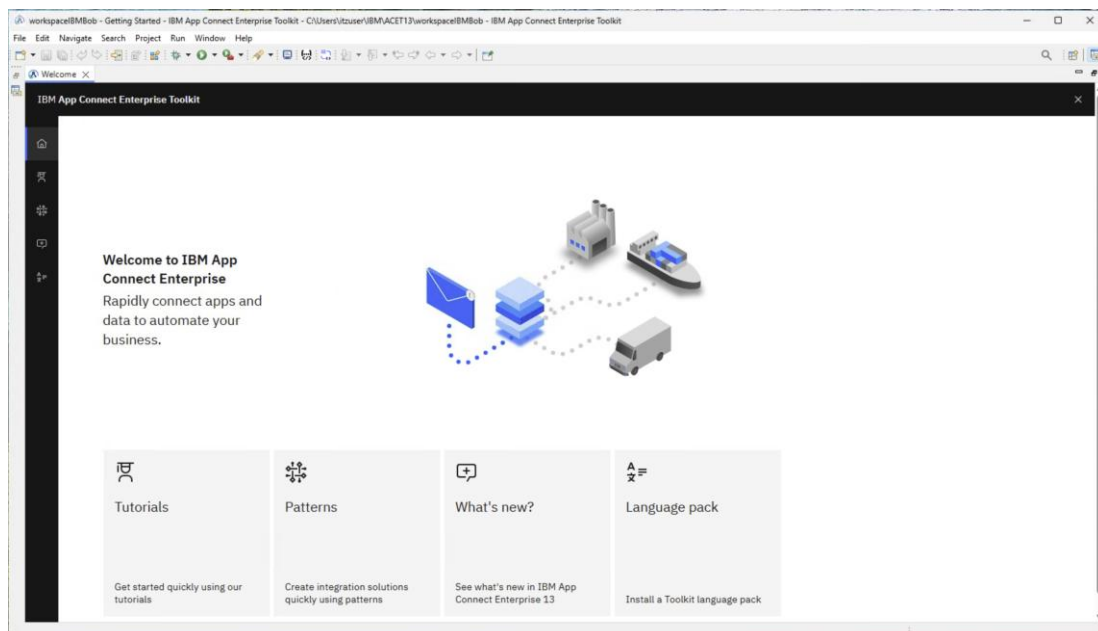
C:\Users\itzuser\IBM\ACET13\workspaceIBMBob



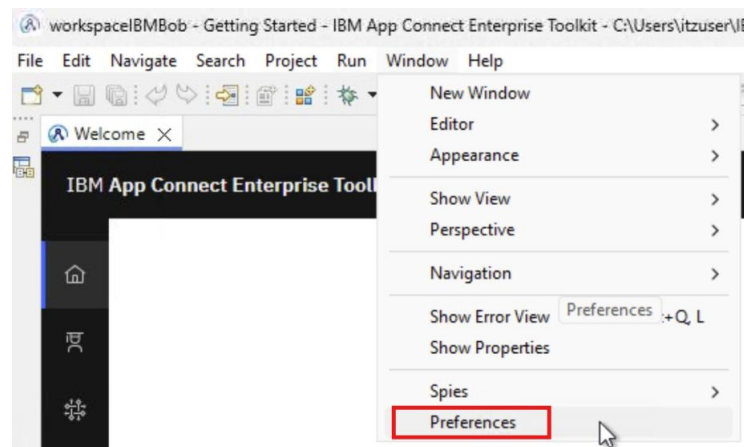
8. If this is the first time you have used the ACE Toolkit on your platform (as it will likely be if you are using the IBM provided environment), then when the Toolkit launches you will be asked about excluding the Toolkit from future Microsoft Windows Defender scans. We strongly recommend to exclude otherwise performance of the virtual environment is likely to be very slow:



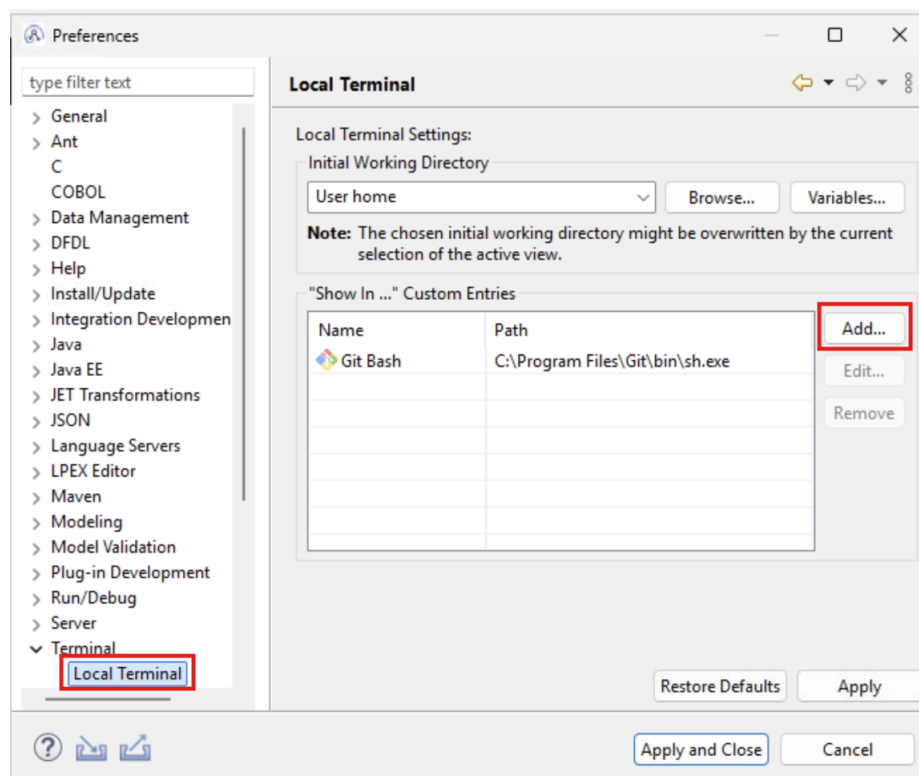
When the Toolkit finishes launching it should look like this:



9. You will now configure the IBM Bob shell inside the Toolkit. From the ACE Toolkit's Window menu choose **Preferences**:

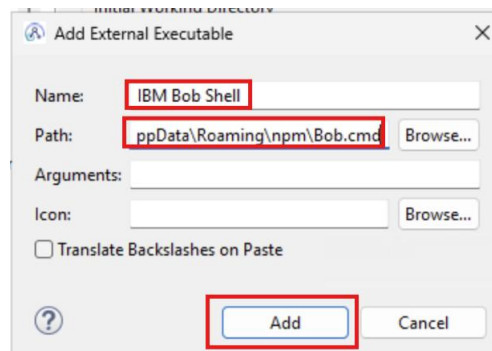


10. When the Preferences pop-up opens, navigate to the section **Terminal > Local Terminal** and click the **Add** button (on the right side of the window):

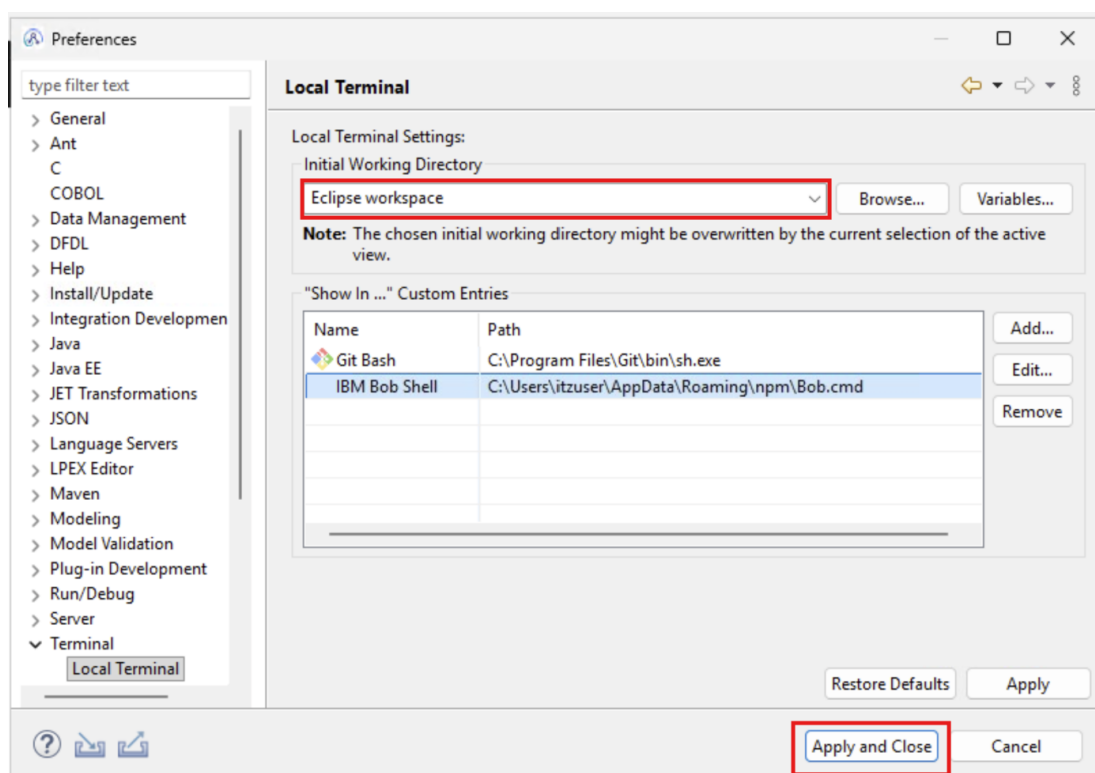


11. In the resulting dialog provide the following details and then click the **Add** button:

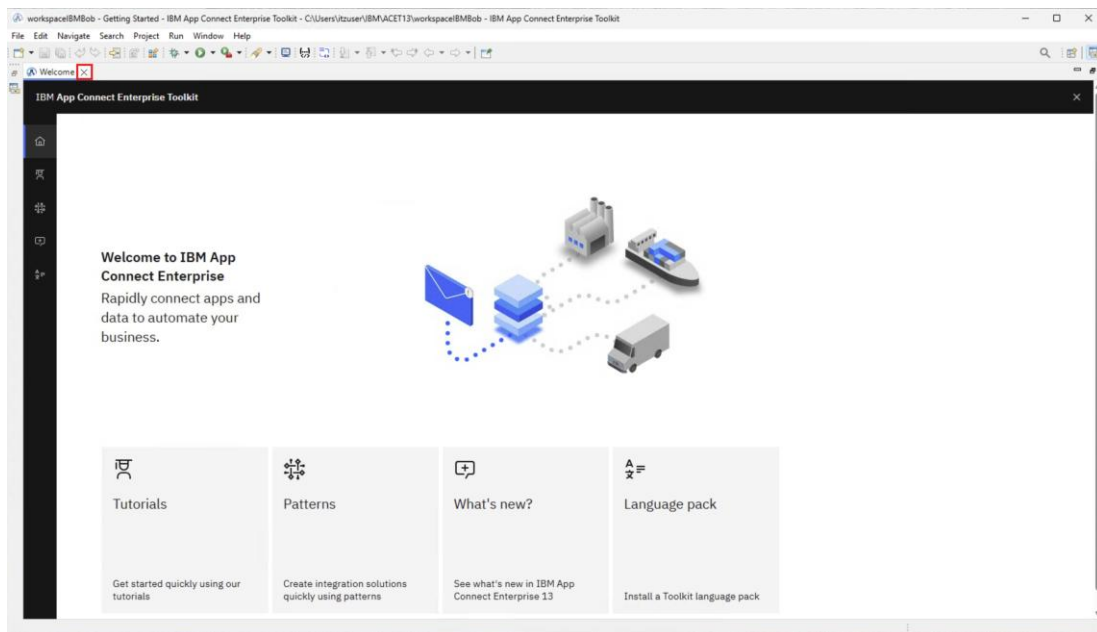
- **Name** = IBM Bob Shell
- **Path** = C:\Users\itzuser\AppData\Roaming\npm\Bob.cmd



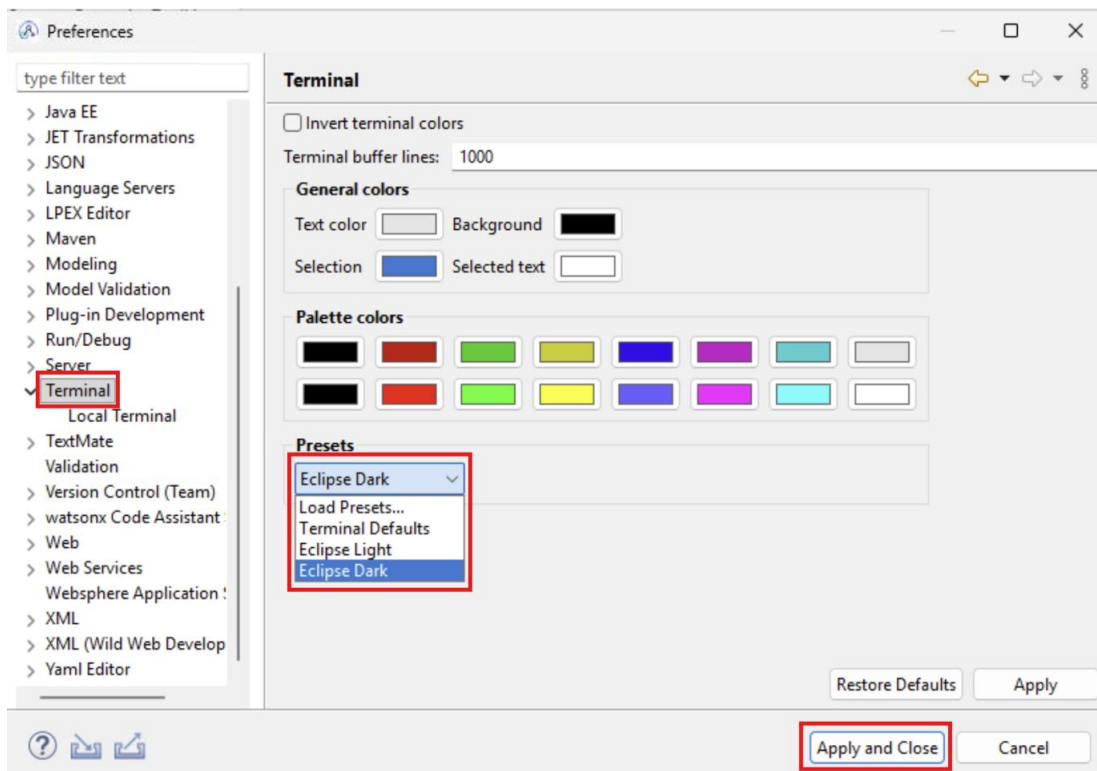
12. Control will return back to the previous **Preferences** window. From the drop-down menu change the **Initial Working Directory** to be Eclipse workspace and then click the **Apply and Close** button.



13. With the dialog closed, you will be returned back to the main view of the Toolkit, where you should click the cross control to close the Welcome page:

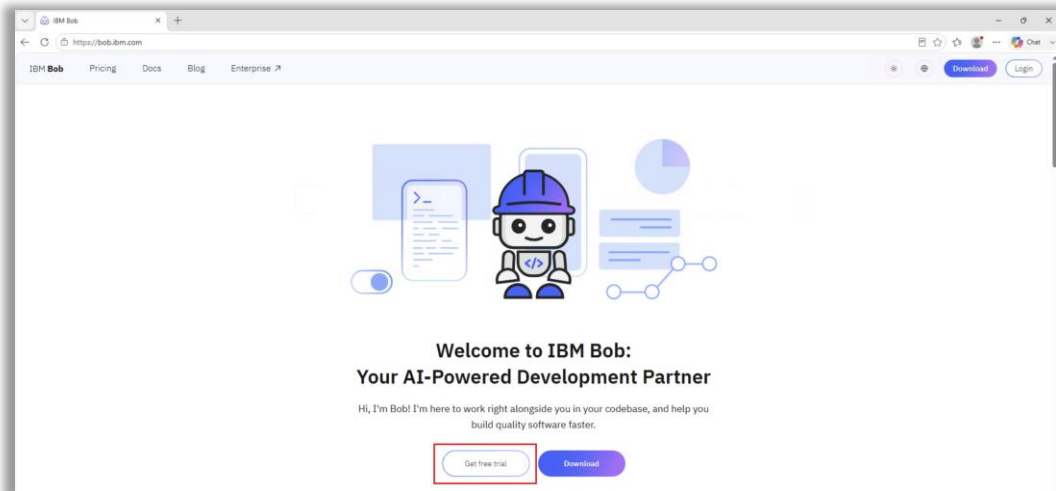


14. From the **Window** menu, choose **Preferences** again. This time, when the **Preferences** window opens, navigate to the section **Terminal**. From the **Presets** drop-down, select **Eclipse Dark** and click the **Apply and Close** button:



4. Register for the IBM Bob trial

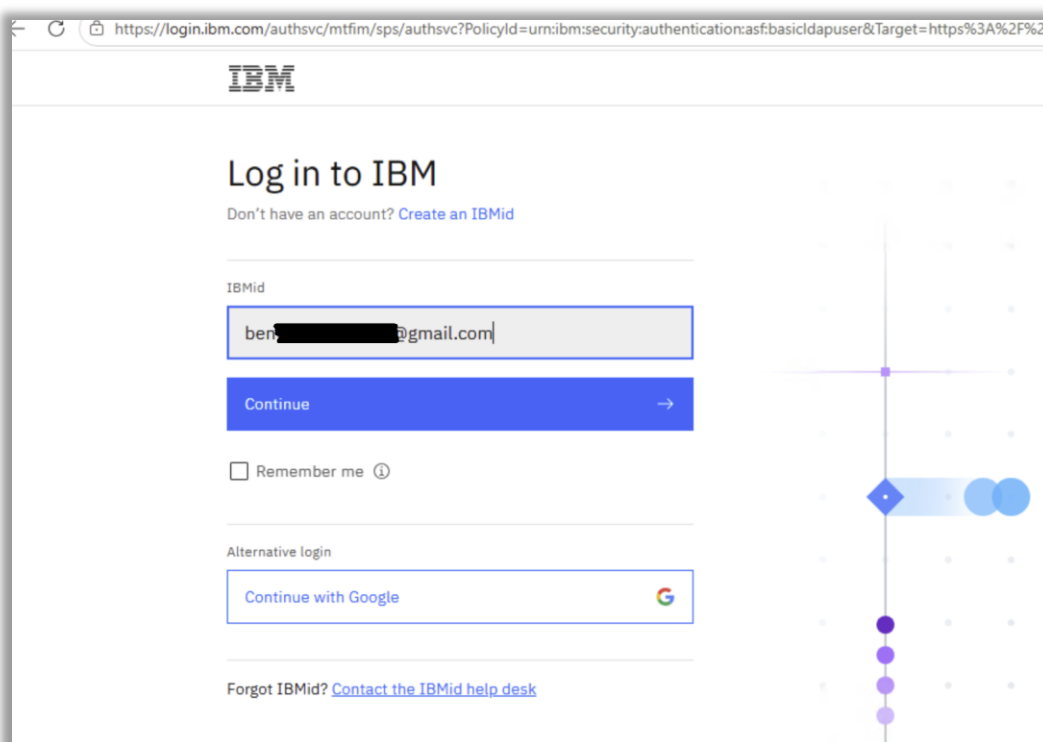
15. Open a web browser and navigate to the URL <https://bob.ibm.com> When the page loads, click the button to **Get Free Trial**:



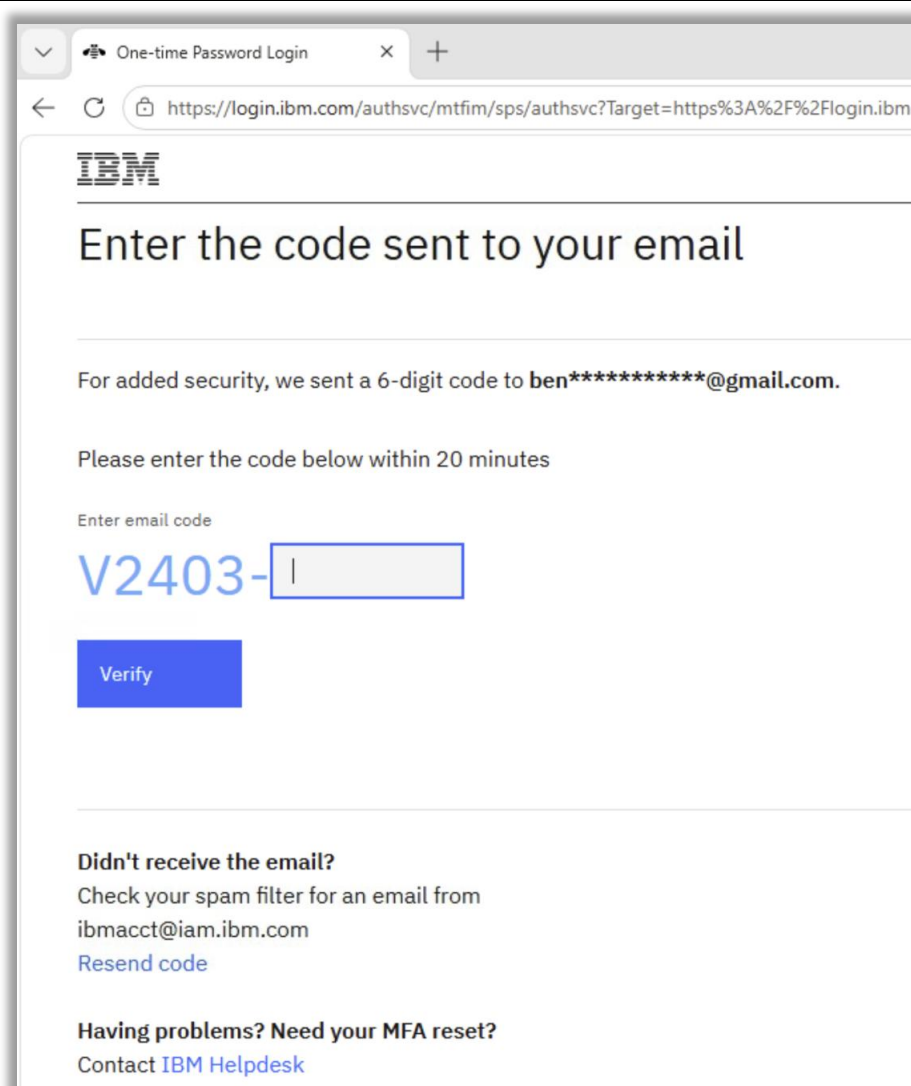
16. The Start Your Free Trial page will load. You can fill out your details if you have never created an IBM account before, but more likely, you can just click the **Log in** link highlighted below:

A screenshot of the 'Start Your Free Trial' registration page. The heading is 'Start Your Free Trial' with a sub-headline 'Experience the full power of IBM Bob: Get 40 Bobcoins for 30 days'. There is a 'Start free trial with Google' button. Below this is a section titled 'Register for your trial' with a link 'Already have an IBM account? Log in' (highlighted with a red box). The form is divided into 'Account information' and a section for personal details. Fields include 'Email', 'Password', 'First name', 'Last name', 'Country or region of residence' (set to 'United States of America'), and 'Company (optional)'. A 'Next' button is at the bottom. A disclaimer at the bottom states: 'During the trial, we may share guidance on which features to explore. We may also reach out to you to check in on your progress and follow up after the trial is over. By participating in the trial, you agree to the IBM Privacy Statement.'

17. You will be passed to a login page where you should enter your IBMid:



After entering your password, you may also be asked to verify yourself using a code number which has been sent to your email:



One-time Password Login

https://login.ibm.com/authsvc/mtfim/sps/authsvc?Target=https%3A%2F%2Flogin.ibm.com

IBM

Enter the code sent to your email

For added security, we sent a 6-digit code to ben*****@gmail.com.

Please enter the code below within 20 minutes

Enter email code

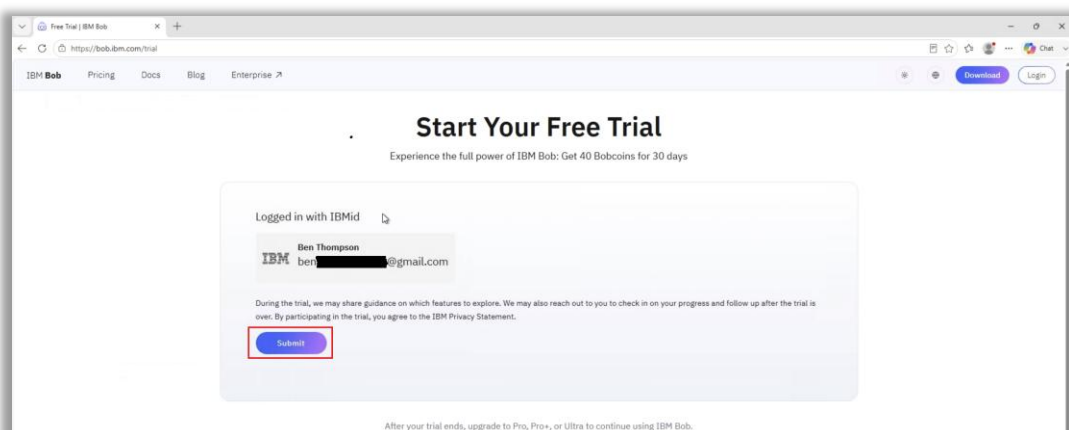
V2403-

Verify

Didn't receive the email?
Check your spam filter for an email from
ibmacct@iam.ibm.com
[Resend code](#)

Having problems? Need your MFA reset?
Contact [IBM Helpdesk](#)

18. After successfully logging in, click the **Submit** button to start your trial:



Free Trial | IBM Bob

https://bob.ibm.com/trial

IBM Bob Pricing Docs Blog Enterprise

Start Your Free Trial

Experience the full power of IBM Bob: Get 40 Bobcoins for 30 days

Logged in with IBMid

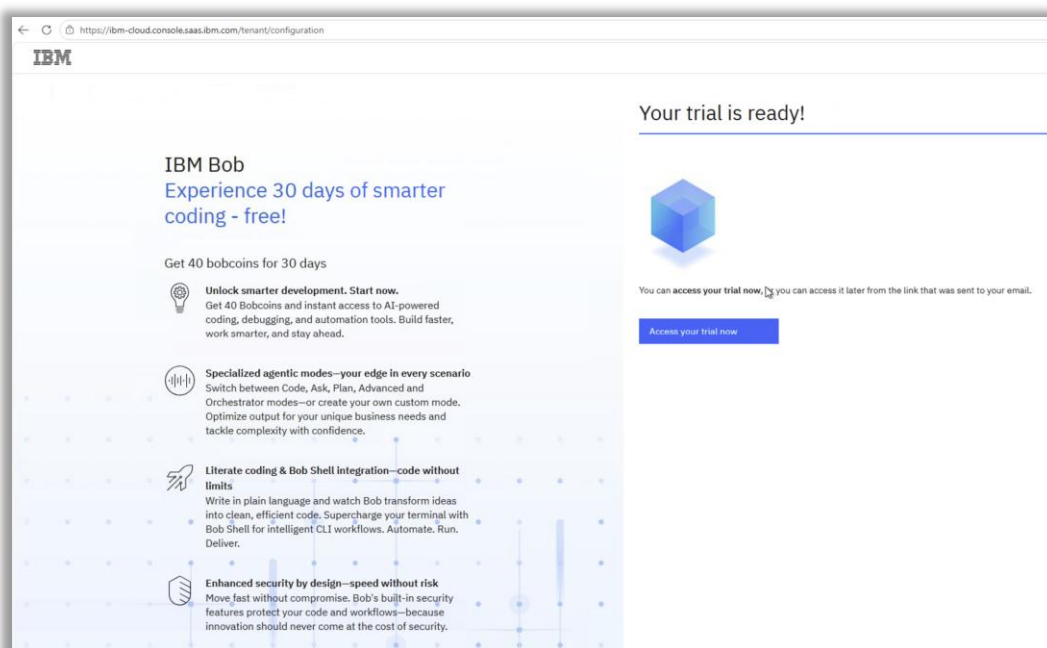
Ben Thompson
ben*****@gmail.com

During the trial, we may share guidance on which features to explore. We may also reach out to you to check in on your progress and follow up after the trial is over. By participating in the trial, you agree to the IBM Privacy Statement.

Submit

After your trial ends, upgrade to Pro, Pro+, or Ultra to continue using IBM Bob.

19. A new page will load confirming that your trial is ready:



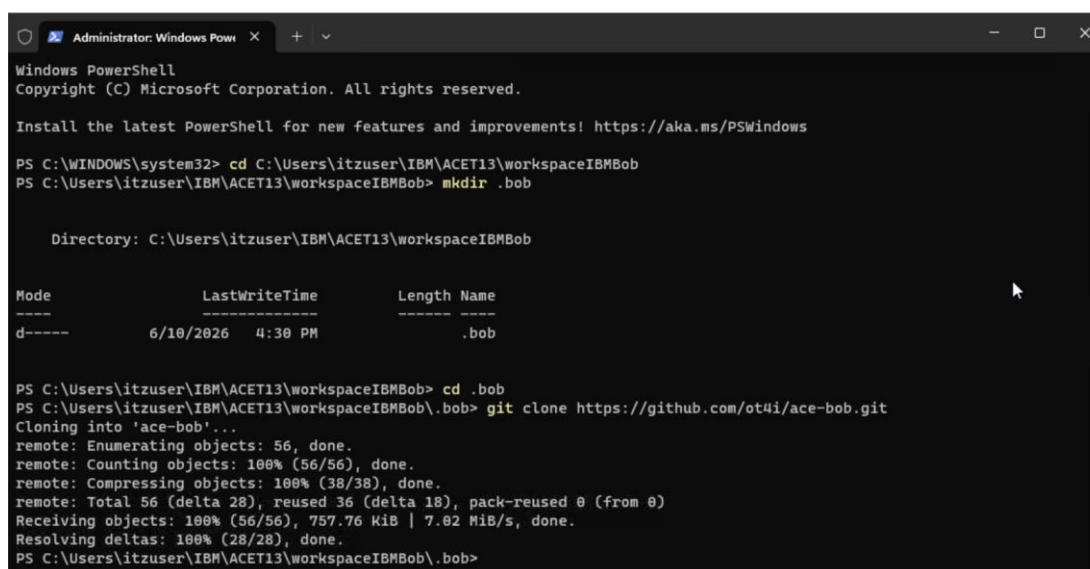
5. Download and configure the ace-bob IBM Bob Skill

20. Skills are reusable instruction sets that teach IBM Bob about specialized tasks so they can be completed in a consistent, repeatable manner. When you activate a skill, IBM Bob receives the skill's instructions and gains access to any supporting files in the skill directory. Bob then follows these instructions to complete your task according to the defined workflow. Skills load once per conversation to avoid duplicate prompts. Bob automatically determines when to activate a skill based on your request and the skill's description.

When operating IBM Bob Shell in the ACE Toolkit, the prior configuration steps documented above have targetted your ACE Toolkit Eclipse workspace to be the IBM Bob project root directory. So, to make skills available for IBM Bob Shell in this context, we will navigate to this ACE Toolkit Eclipse workspace and if it does not already exist, create a folder called `/.bob`. Alternatively if you want to use IBM Bob across multiple ACE Toolkit workspaces, you might prefer to define ACE skills globally in your user's home directory. Based on our experiences, we think the best choice would be to locate the skill in the `.bob` folder within your Toolkit workspace as this improves the chances of Bob being able to find it and use it consistently.

In the windows environment, open a Windows PowerShell console and execute the following commands to clone the GitHub repository which contains the ace-bob skill:

- `cd C:\users\itzuser\IBM\ACET13\workspaceIBMBob`
- `mkdir .bob`
- `cd .bob`
- `git clone https://github.com/ot4i/ace-bob.git`



```

Administrator: Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\WINDOWS\system32> cd C:\Users\itzuser\IBM\ACET13\workspaceIBMBob
PS C:\Users\itzuser\IBM\ACET13\workspaceIBMBob> mkdir .bob

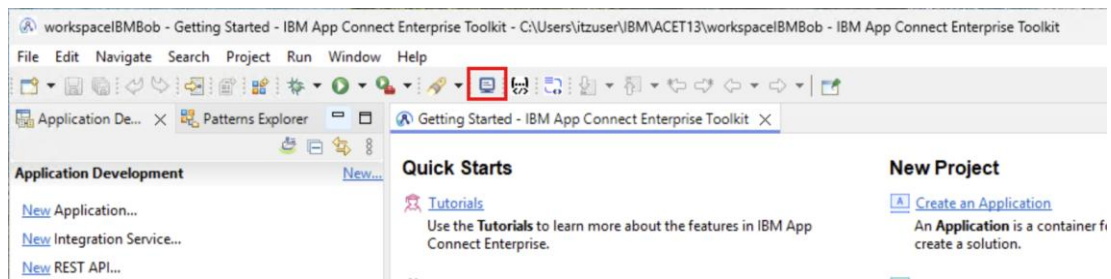
Directory: C:\Users\itzuser\IBM\ACET13\workspaceIBMBob

Mode                LastWriteTime         Length Name
----                -
d-----          6/10/2026   4:30 PM             .bob

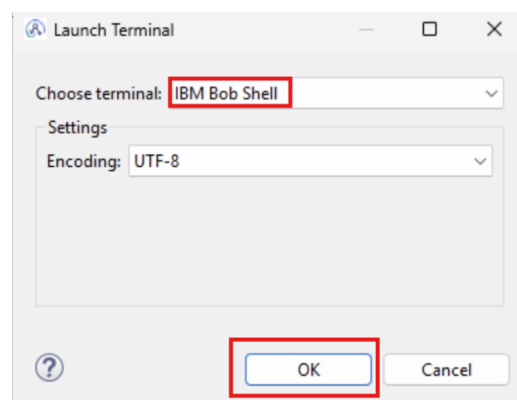
PS C:\Users\itzuser\IBM\ACET13\workspaceIBMBob> cd .bob
PS C:\Users\itzuser\IBM\ACET13\workspaceIBMBob\.bob> git clone https://github.com/ot4i/ace-bob.git
Cloning into 'ace-bob'...
remote: Enumerating objects: 56, done.
remote: Counting objects: 100% (56/56), done.
remote: Compressing objects: 100% (38/38), done.
remote: Total 56 (delta 28), reused 36 (delta 18), pack-reused 0 (from 0)
Receiving objects: 100% (56/56), 757.76 KiB | 7.02 MiB/s, done.
Resolving deltas: 100% (28/28), done.
PS C:\Users\itzuser\IBM\ACET13\workspaceIBMBob\.bob>
  
```

6. Use IBM Bob to create a simple HTTP flow

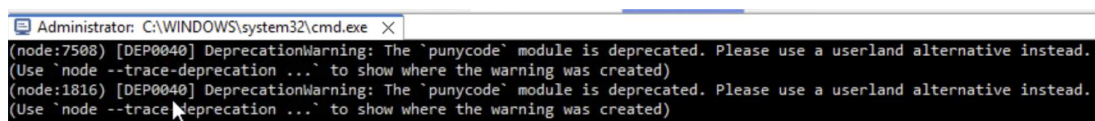
21. Return to the ACE Toolkit, as we are now ready to launch the IBM Bob Shell inside the Toolkit. From the top Toolbar, click the **Terminal** shortcut (shown in the red box below):



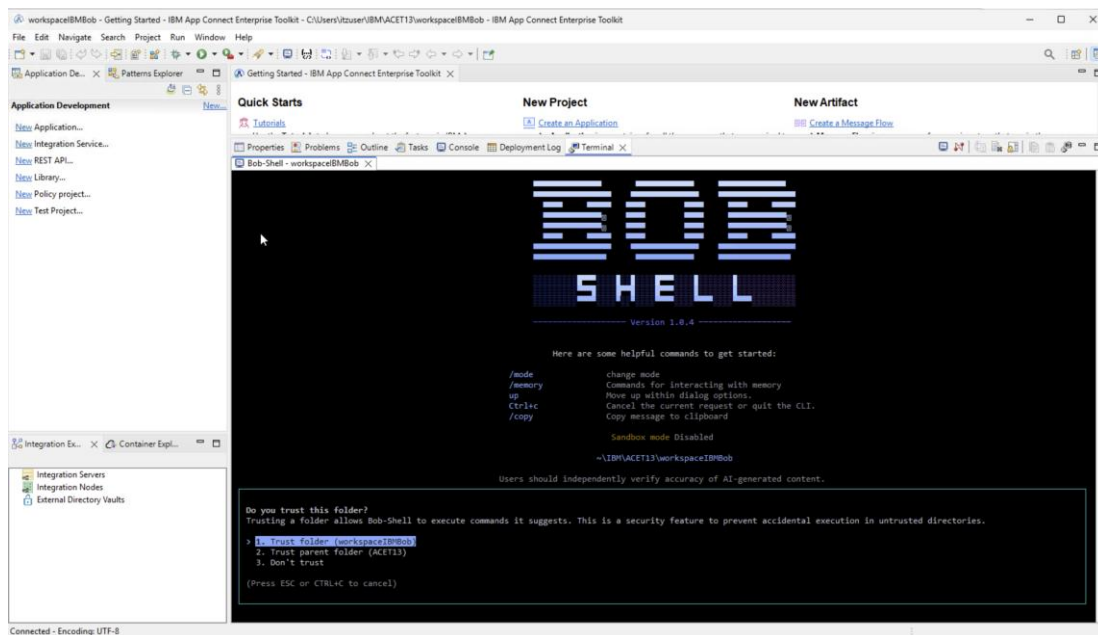
22. From the Launch Terminal pop-up, select **IBM Bob Shell** from the **Choose terminal** dropdown and click **OK**:



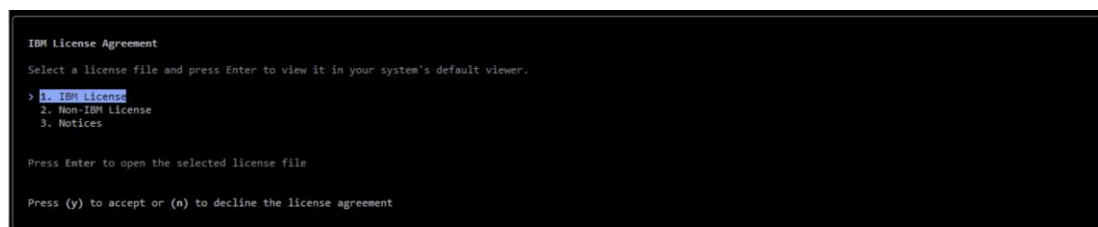
23. The IBM Bob Shell terminal will start to load (typically it will take about 10-20 seconds to load). During this time you may see a deprecation warning message appear relating to some nodeJS pre-requisites – this will depend on the version of IBM Bob which you installed and the version of nodeJS which you are running. An example of this is shown below, but you may see something slightly different. Do not be concerned, just wait a few more seconds and the shell will continue to load.



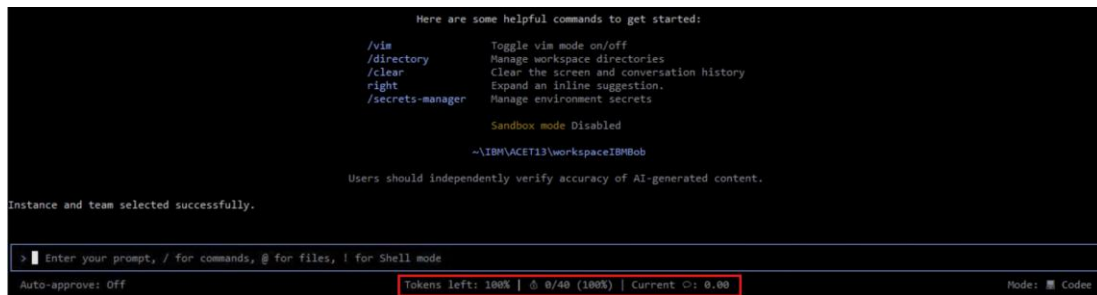
24. An IBM Bob Shell will open (by default in the bottom right corner of the screen. Depending on the size and arrangement of your views in Eclipse, it may look slightly different to the picture below (the 8-bar BOB logo may have scrolled off the top of the terminal window for example). You may also find that a redirection causes a web browser window to open in front of the ACE Toolkit asking you for IBMid authentication (if this happens, fill in your IBMid details just like you did earlier in the Lab). The first thing that Bob will ask is whether you agree to trust Bob's execution in your workspace. Agree to trust the folder by pressing the Enter key whilst the option is highlighted (which it will be by default):



25. Next you will be shown a License agreement which you must accept in order to continue with the Lab. To open the license file, with the option highlighted press the Enter key on your keyboard. The license may open with different applications depending on your system defaults. If you are running this lab in the IBM provided environment then it will open in Notepad. After reading the license, to accept the license, press the "y" key on your keyboard:



26. If this is the first time you have used your IBM Bob trial, you will see that at the bottom of the page (beneath the prompt, shown in the red box below) there is a reminder about how many Bob coins you have spent (on the picture below 0 out of 40 as this screenshot was taken before using the trial):



```
Here are some helpful commands to get started:

/via          Toggle via mode on/off
/directory    Manage workspace directories
/clear        Clear the screen and conversation history
/right        Expand an inline suggestion.
/secrets-manager Manage environment secrets

Sandbox mode Disabled

~\IBM\ACET13\workspace\IBMBob

Users should independently verify accuracy of AI-generated content.

Instance and team selected successfully.

> Enter your prompt, / for commands, @ for files, ! for Shell mode

Auto-approve: Off Tokens left: 100% | 0/40 (100%) | Current 0: 0.00 Mode: Codee
```

27. We are now ready to try out some prompts. The first prompt we will use will show how the skill that we applied can be used to help Bob create ACE Projects that have all of the relevant Eclipse metadata inside them so that they render correctly without any errors or warnings when you import and use them. We will start with a request to create a very simple message flow. Please note that due to the non-deterministic nature of AI, there is no **guarantee** that you will see exactly the same results when you try out these examples. However, if you find that your results are *very* different from the results shown, then please get your instructor to take a look and pass comment. In the IBM Bob Shell type out the following prompt. You will have an opportunity to try your own prompts later in the lab, but please start off using the prompt word-for-word as written below so that you get the best chance possible of seeing the Bob skill in action (if you don't specifically tell IBM Bob to use the skill there is a greater chance that Bob will go seeking out other flow examples rather than take on board the advice in the skill):

Use the ace-bob skill to help create me an ACE message flow which takes in HTTP content and echoes it back to the requester. Create this flow in an application called Echo.

```
> Use the ace-bob skill to help create me an ACE message flow which takes in HTTP content and echoes it back to the requester. Create this flow in an application called Echo
```

28. IBM Bob will start considering the best way to answer your question. You will see Bob describing these considerations, and at various stages IBM Bob will pause whilst seeking confirmations from you:

```

Bob-Shell - workspace\BMBob X
<thinking>
Analyzing task to create ACE message flow

The user wants me to create an ACE message flow that:
1. Takes in HTTP content
2. Echoes it back to the requester
3. Should be in an application called "Echo"

Based on the SKILL.md documentation, I need to:
1. Create an ACE Toolkit Application Project named "Echo"
2. Create the .project file with proper natures and buildSpec
3. Create .settings subdirectory with org.eclipse.core.resources.prefs
4. Create application.descriptor file
5. Create a message flow (.msgflow) that uses HTTP Input and HTTP Reply nodes

This is a straightforward task with clear requirements. Let me start by creating the project structure.
</thinking>

% To-do List Updating 5 item(s)
1
2 - [ ] Create Echo application project directory structure
3 - [ ] Create .project file with ACE Application project natures
4 - [ ] Create .settings/org.eclipse.core.resources.prefs file
5 - [ ] Create application.descriptor file
6 - [ ] Create Echo.msgflow with HTTP Input and HTTP Reply nodes

Apply this change?
> 1. Yes, allow once
2. Yes, allow always
3. Modify with external editor: none (select one with /editor command)
4. No, suggest changes (esc)

Waiting for user confirmation...

Auto-approve: Off Tokens left: 84% | 0.11/40 (99%) | Current 0: 0.11 Mode: Code

```

In the picture above you can see that IBM Bob has formulated a To-do List for answering the question. The list involves creating an ACE application project and then populating it with three Eclipse metadata files:

- .project file
- .settings/org.eclipse.core.resources.prefs file
- application.descriptor file

The last task in the list is to create the message flow file in the project.

29. After agreeing to let IBM Bob continue, in this example (remember you may see slight differences), IBM Bob recognised the need to start by creating the application directory called **Echo**:

```

% To-do List Updating 5 item(s)

[ ] Create Echo application project directory structure
[ ] Create .project file with ACE Application project natures
[ ] Create .settings/org.eclipse.core.resources.prefs file
[ ] Create application.descriptor file
[ ] Create Echo.msgflow with HTTP Input and HTTP Reply nodes

<thinking>Create Echo application directory
I need to create the Echo application project structure. First, I'll create the main Echo directory.
</thinking>

Shell New-Item -ItemType Directory -Path "Echo" -Force [current working directory C:\Users\ituser\IBM\ACET13\workspace\BMBob] (10s)

New-Item -ItemType Directory -Path "Echo" -Force

Confirm Shell Command
Allow execution of: 'New-Item'?

> 1. Yes, allow once
2. Yes, allow always ...
3. No, suggest changes (esc)

Waiting for user confirmation...

```

30. After agreeing that IBM Bob should continue, the next task is to create a .project file:

```
Directory: C:\Users\ituser\IBM\ACET13\workspace\IBMBob

Mode                LastWriteTime         Length Name
----                -
d-----          6/11/2026   1:40 PM             Echo

B <thinking>Create .project file
Directory created successfully. Now I need to create the .project file with the correct ACE Application project natures and build commands as specified in the skill
documentation.
</thinking>

B Create .project file (esc to cancel, 20s)
```

A few seconds later, the picture below shows the structure of the .project file, which because it is an ACE application project, it needs two different natures, which you can see at the end of the generated XML:

```
WriteFile Writing to Echo\.project

... first 103 lines hidden ...
104     <name>com.ibm.etools.mft.bar.ext.barbuilder</name>
105     <arguments>
106     </arguments>
107   </buildCommand>
108   <buildCommand>
109     <name>com.ibm.etools.mft.unittest.ui.TestCaseBuilder</name>
110     <arguments>
111     </arguments>
112   </buildCommand>
113 </buildSpec>
114 <natures>
115   <nature>com.ibm.etools.msgbroker.tooling.applicationNature</nature>
116   <nature>com.ibm.etools.msgbroker.tooling.messageBrokerProjectNature</nature>
117 </natures>
118 </projectDescription>

Apply this change?
> 1. Yes, allow once
   2. Yes, allow always
   3. Modify with external editor: none (select one with /editor command)
   4. No, suggest changes (esc)

Press ctrl-s to show more lines
@ Waiting for user confirmation...
```

31. Next, IBM Bob knows to create a `.settings` subdirectory in the project which will contain the preferences file:

```
<thinking>Creating .settings directory and prefs file
Need to create the .settings subdirectory and the org.eclipse.core.resources.prefs file inside it.
</thinking>

Shell New-Item -ItemType Directory -Path "Echo\.settings" -Force [current working directory C:\Users\itzuser\IBM\ACET13\workspace\IBMBob] (10s)

New-Item -ItemType Directory -Path "Echo\.settings" -Force

Confirm Shell Command
Allow execution of: 'New-Item'?

> 1. Yes, allow once
   2. Yes, allow always ...
   3. No, suggest changes (esc)

Waiting for user confirmation...
```

The content of the preferences file is generated and IBM Bob asks for permission to apply the change as shown:

```
<thinking>Creating settings file
Need to create the org.eclipse.core.resources.prefs file in the .settings directory with UTF-8 encoding configuration.
</thinking>

% WriteFile Writing to org.eclipse.core.resources.prefs

1 eclipse.preferences.version=1
2 encoding/<project>=UTF-8

Apply this change?

> 1. Yes, allow once
   2. Yes, allow always
   3. Modify with external editor: none (select one with /editor command)
   4. No, suggest changes (esc)

Waiting for user confirmation...
```

32. Next, IBM Bob knows to create an `application.descriptor` file in the project:

```
<thinking>Creating application.descriptor file
Next step is to create the application.descriptor file in the Echo project root directory.
</thinking>

% WriteFile Writing to Echo\application.descriptor

1 <?xml version="1.0" encoding="UTF-8" standalone="yes"?>
2 <ns2:appDescriptor xmlns="http://com.ibm.etools.mft.descriptor.base" xmlns:ns2="http://com.ibm.etools.mft.descriptor.app">
3   <references/>
4 </ns2:appDescriptor>

Apply this change?

> 1. Yes, allow once
   2. Yes, allow always
   3. Modify with external editor: none (select one with /editor command)
   4. No, suggest changes (esc)

Waiting for user confirmation...
```

33. The last step is to create the message flow itself:

```
<thinking>Creating Echo message flow
Now I need to create the Echo.msgflow file with HTTP Input and HTTP Reply nodes connected together to echo back the HTTP content.
</thinking>

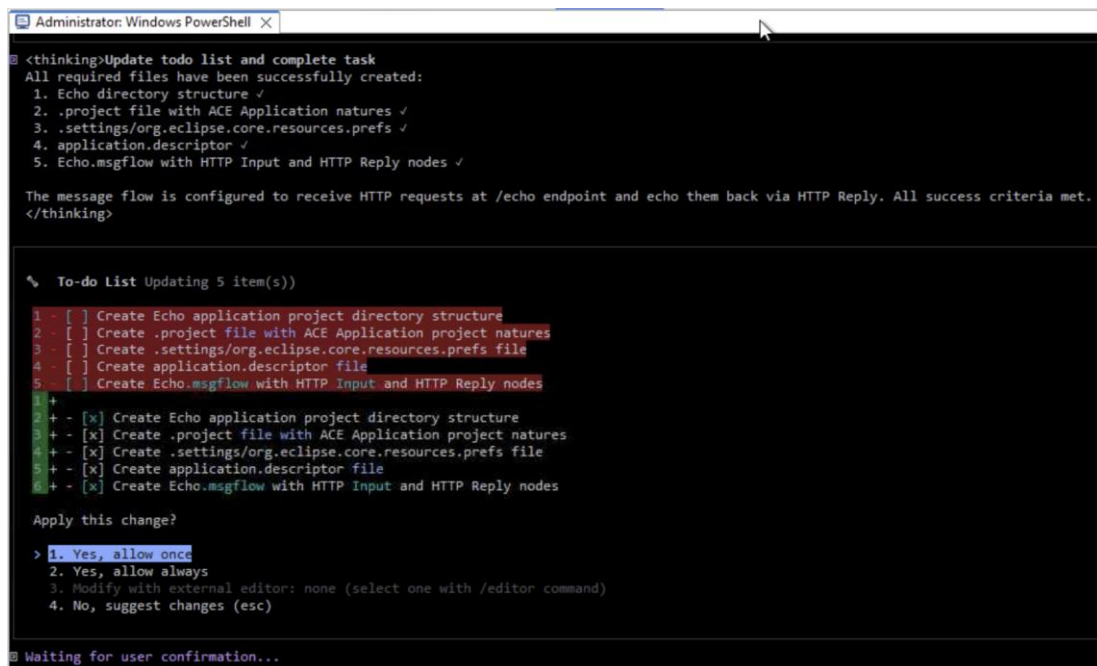
% WriteFile Writing to Echo\Echo.msgflow

... first 6 lines hidden ...
7 <colorGraphic32 xmi:type="utility:GIFFileGraphic" resourceName="platform:/plugin/Echo/icons/full/obj30/Echo.gif"/>
8 <composition>
9   <nodes xmi:type="ComIbmMInput.msgnode:FCMComposite_1" xmi:id="FCMComposite_1_1" location="100,100" URLSpecifier="/echo">
10     <translation xmi:type="utility:ConstantString" string="HTTP Input"/>
11   </nodes>
12   <nodes xmi:type="ComIbmMReply.msgnode:FCMComposite_1" xmi:id="FCMComposite_1_2" location="300,100">
13     <translation xmi:type="utility:ConstantString" string="HTTP Reply"/>
14   </nodes>
15   <connections xmi:type="eflow:FCMConnection" xmi:id="FCMConnection_1" targetNode="FCMComposite_1_2" sourceNode="FCMComposite_1_1" sourceTerminalName=
"OutTerminal.out" targetTerminalName="InTerminal.in"/>
16 </composition>
17 <propertyOrganizer/>
18 <stickyBoard/>
19 </eClassifier/>
20 </ecore:EPackage>

Apply this change?
> 1. Yes, allow once
2. Yes, allow always
3. Modify with external editor: none (select one with /editor command)
4. No, suggest changes (esc)

Press ctrl-s to show more lines
Waiting for user confirmation...
```

34. IBM Bob then goes back to the initial To-do list to check on the progress overall:



```
Administrator: Windows PowerShell X
<thinking>Update todo list and complete task
All required files have been successfully created:
1. Echo directory structure ✓
2. .project file with ACE Application natures ✓
3. .settings/org.eclipse.core.resources.prefs ✓
4. application.descriptor ✓
5. Echo.msgflow with HTTP Input and HTTP Reply nodes ✓

The message flow is configured to receive HTTP requests at /echo endpoint and echo them back via HTTP Reply. All success criteria met.
</thinking>

To-do List Updating 5 item(s)

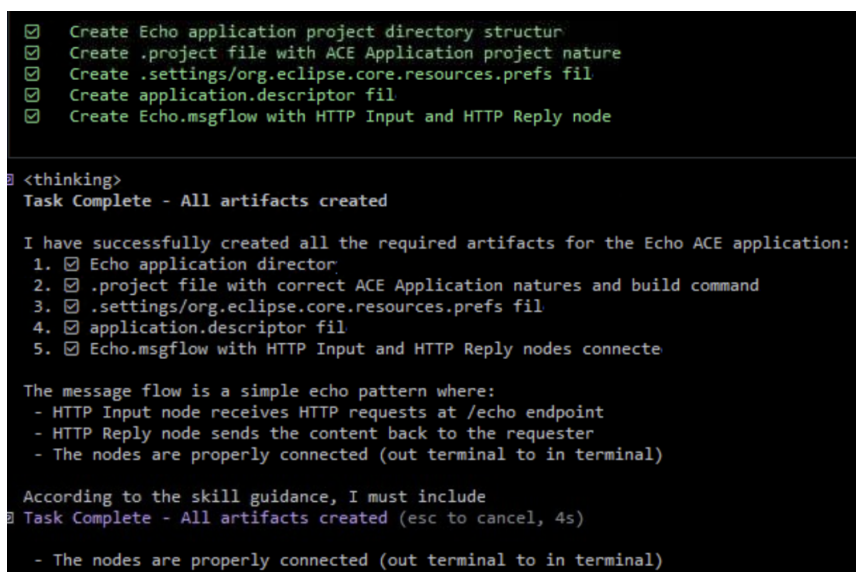
1 - [ ] Create Echo application project directory structure
2 - [ ] Create .project file with ACE Application project natures
3 - [ ] Create .settings/org.eclipse.core.resources.prefs file
4 - [ ] Create application.descriptor file
5 - [ ] Create Echo.msgflow with HTTP Input and HTTP Reply nodes

1 +
2 + - [X] Create Echo application project directory structure
3 + - [X] Create .project file with ACE Application project natures
4 + - [X] Create .settings/org.eclipse.core.resources.prefs file
5 + - [X] Create application.descriptor file
6 + - [X] Create Echo.msgflow with HTTP Input and HTTP Reply nodes

Apply this change?
> 1. Yes, allow once
2. Yes, allow always
3. Modify with external editor: none (select one with /editor command)
4. No, suggest changes (esc)

Waiting for user confirmation...
```

IBM Bob ticks off the items and confirms the message flow nodes that have been placed into the flow and the purpose of each node:



```

[✓] Create Echo application project directory structur
[✓] Create .project file with ACE Application project nature
[✓] Create .settings/org.eclipse.core.resources.prefs fil
[✓] Create application.descriptor fil
[✓] Create Echo.msgflow with HTTP Input and HTTP Reply node

<thinking>
Task Complete - All artifacts created

I have successfully created all the required artifacts for the Echo ACE application:
1. [✓] Echo application director
2. [✓] .project file with correct ACE Application natures and build command
3. [✓] .settings/org.eclipse.core.resources.prefs fil
4. [✓] application.descriptor fil
5. [✓] Echo.msgflow with HTTP Input and HTTP Reply nodes connecte

The message flow is a simple echo pattern where:
- HTTP Input node receives HTTP requests at /echo endpoint
- HTTP Reply node sends the content back to the requester
- The nodes are properly connected (out terminal to in terminal)

According to the skill guidance, I must include
Task Complete - All artifacts created (esc to cancel, 4s)
- The nodes are properly connected (out terminal to in terminal)
```

35. In the final summary IBM Bob also includes some advice from the Skill about what to do next. It reads: "If you are working in the ACE Toolkit then to see the results and do further work with the generated project in the Application Development view, use the menu option to File > Import > Existing Project into workspace."

This is exactly what we will do in the next step.

```
Successfully created the Echo ACE application with a message flow that echoes HTTP content back to the requester.

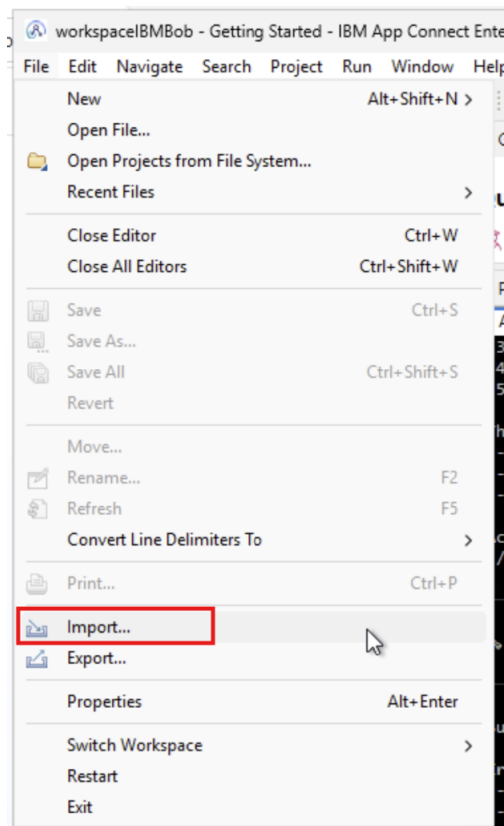
Created artifacts:
- Echo application project directory structure
- .project file with ACE Application project natures and build specifications
- .settings/org.eclipse.core.resources.prefs file with UTF-8 encoding
- application.descriptor file
- Echo.msgflow with HTTP Input node (endpoint: /echo) connected to HTTP Reply node

The message flow receives HTTP requests at the /echo endpoint and immediately returns the received content back to the requester through the HTTP Reply node.

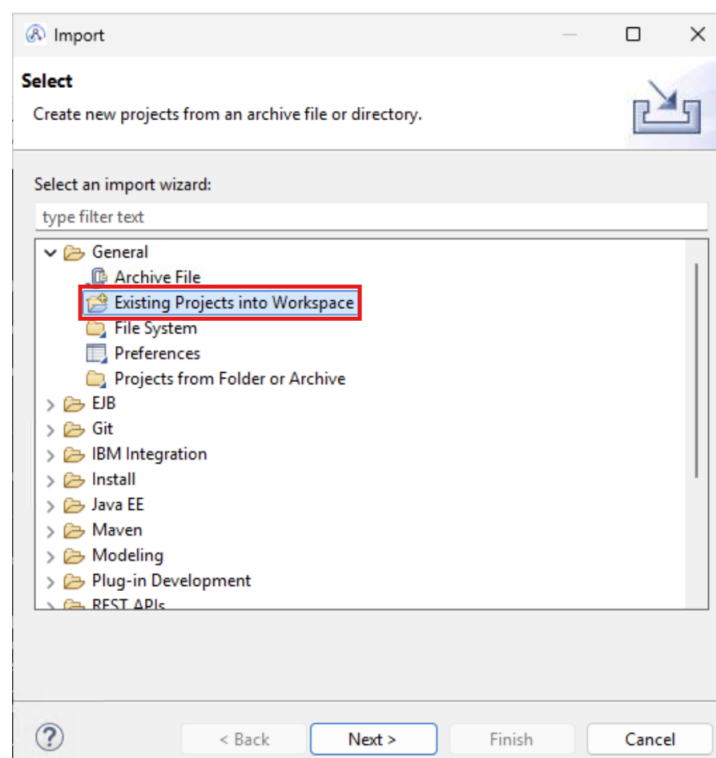
Next Steps:
If you are working in the ACE Toolkit then to see the results and do further work with the generated project in the Application Development view, use the menu option to
File > Import > Existing Projects into Workspace.

You can ask follow up questions or use the /clear command to start a new task
```

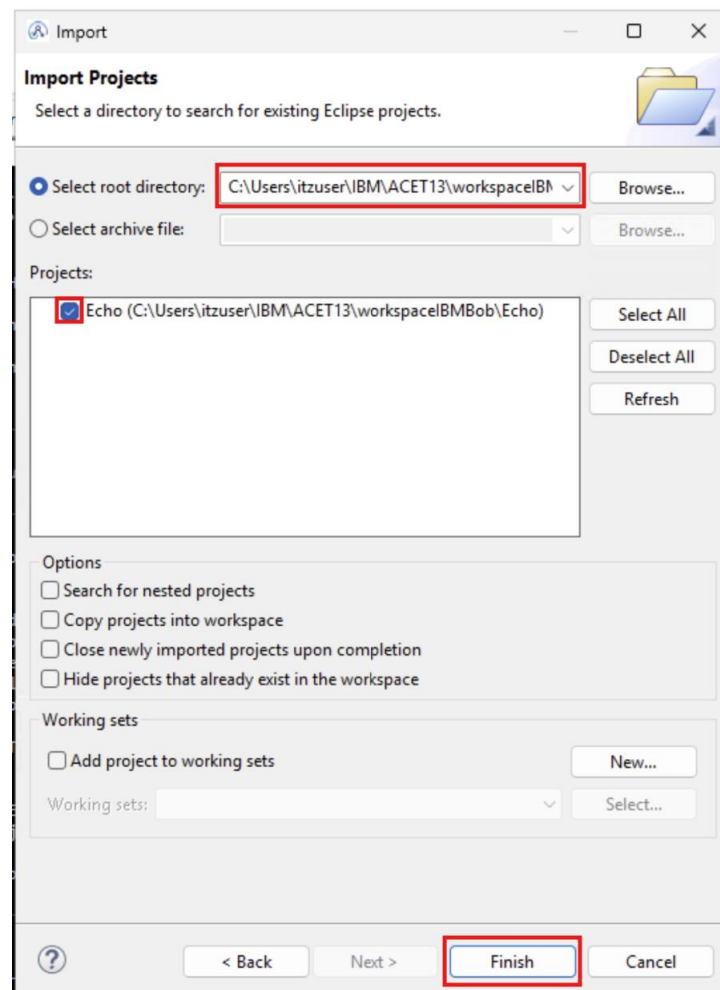
36. IBM Bob has created the Eclipse ACE Toolkit project with all the relevant files but it has not yet been imported into the Eclipse workspace. Go to the menu option **File > Import**:



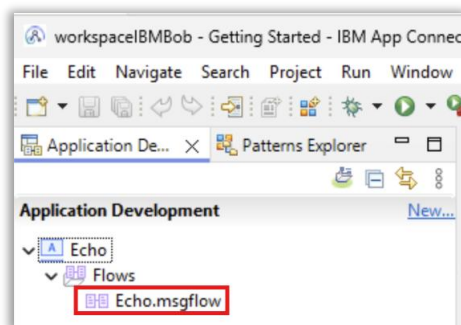
From the Import pop-up select the option to import **Existing Projects into Workspace** and then click **Next**:



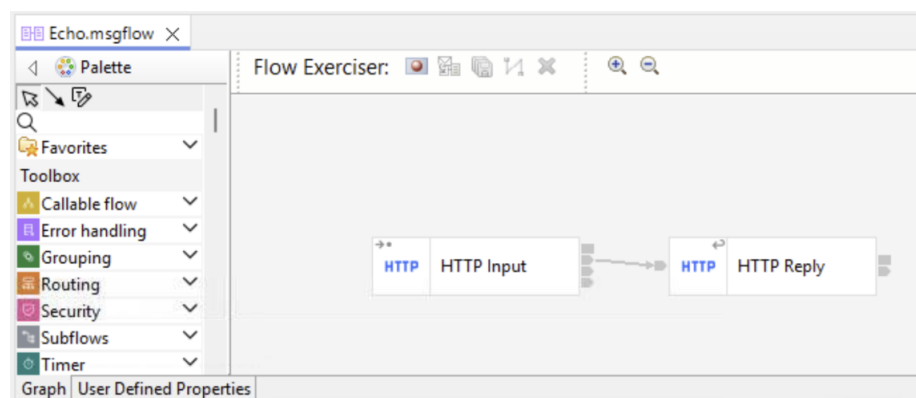
On the next page of the dialog, use the Browse button to navigate to the root directory of C:\Users\itzuser\IBM\ACET13\workspaceIBMBob, then select the **Echo** Project and then click **Finish**:



37. The Application project will be imported and become visible in the top left corner in the Application Development View. Expand the project and then double click **Echo.msgflow** in order to open it:



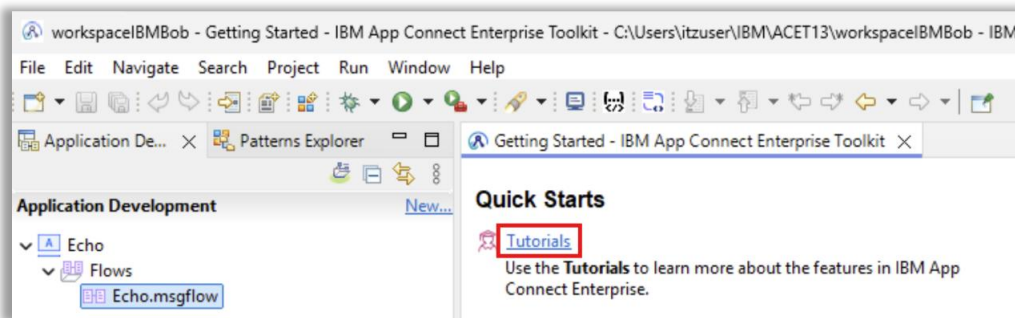
You should find that the Echo message flow has correctly wired an HTTPInput node to an HTTPReply node:



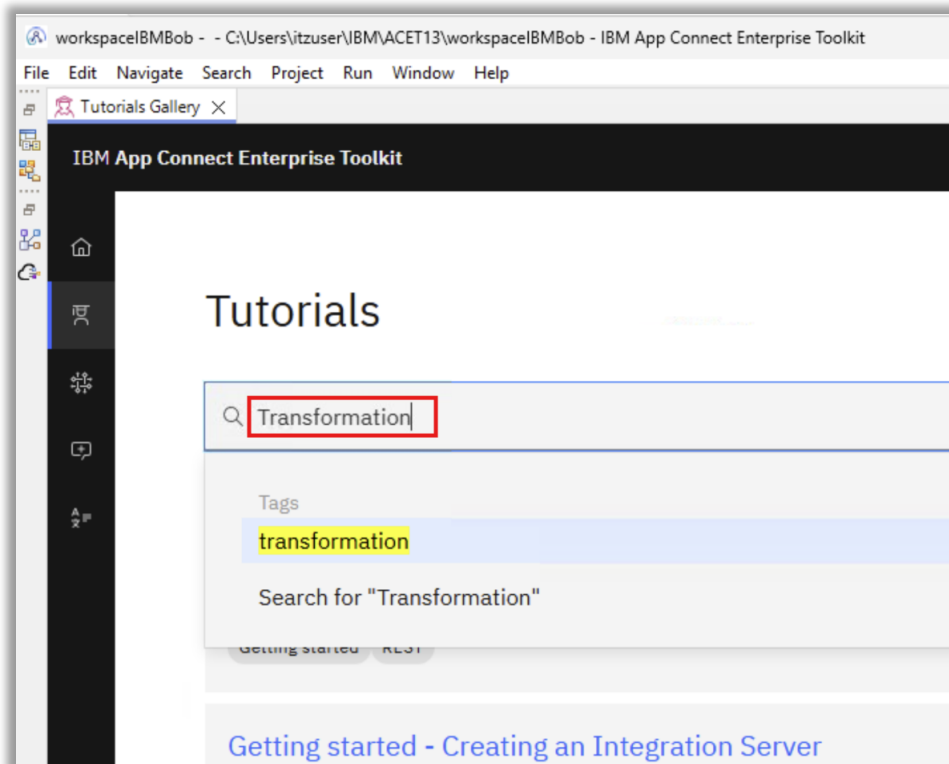
Take a look at the message flow nodes' properties and then close the message flow.

7. Use IBM Bob to summarize an existing flow

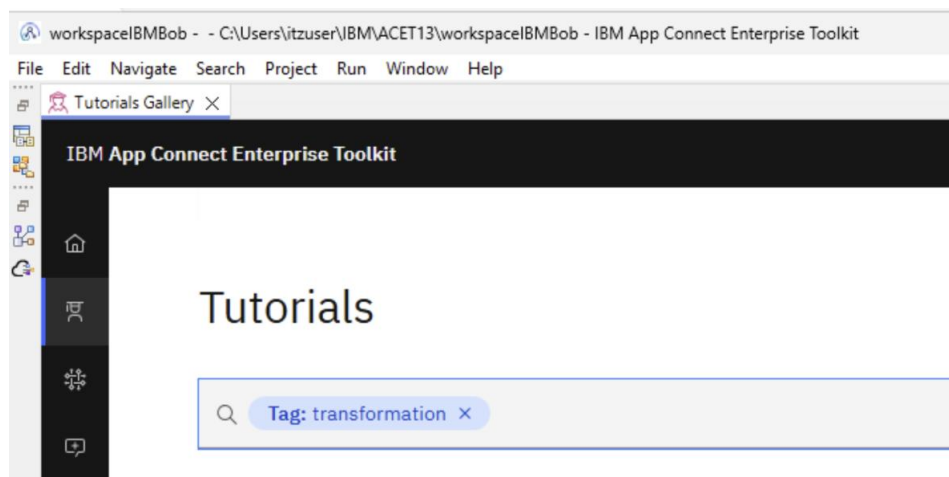
38. We are now ready to try a second example. This time we will get IBM Bob to examine an existing message flow and summarize the transformation tasks that it provides. We'll start by importing an existing message flow and then we will ask IBM Bob to analyze it. We will use one of the ACE Toolkit Tutorials for our starting point. If you closed the message flow in the last step then you should see the Getting Started page in the Toolkit. Click the Tutorials link (if you accidentally closed the page, you can also access the tutorials from the menu option **Help > Tutorials Gallery**):



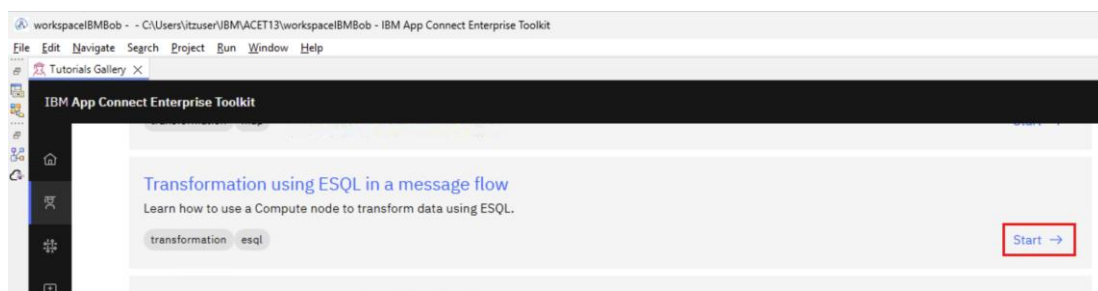
39. When the Tutorials Gallery launches, type Transformation in the filter box and hit return:



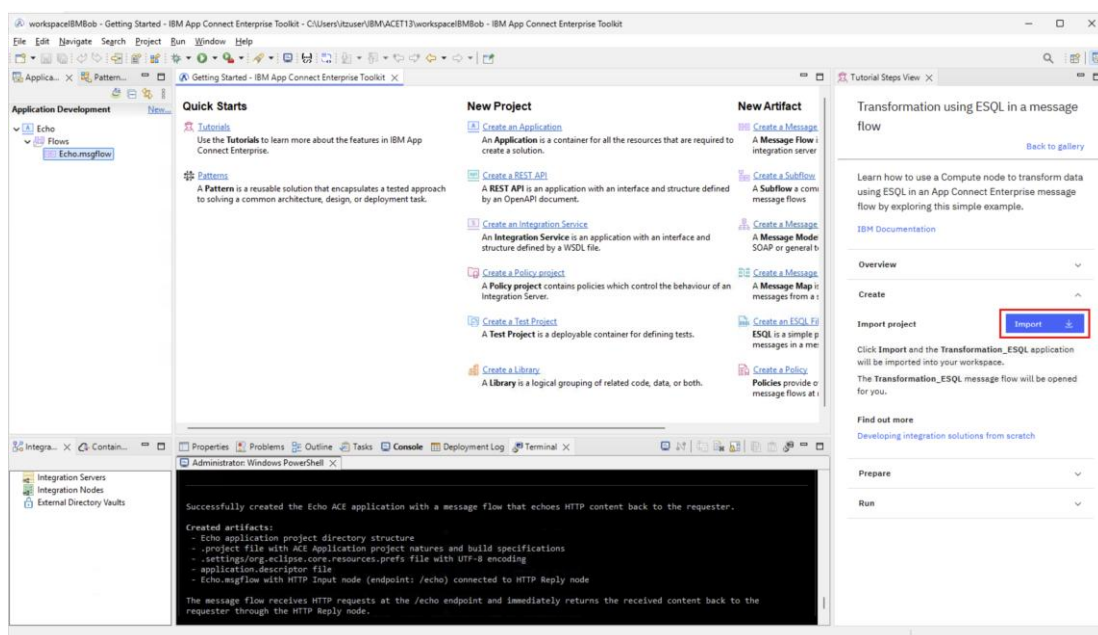
The text will turn into a Tag and beneath the filter you will be left with a set of tutorials that involve transformation code.



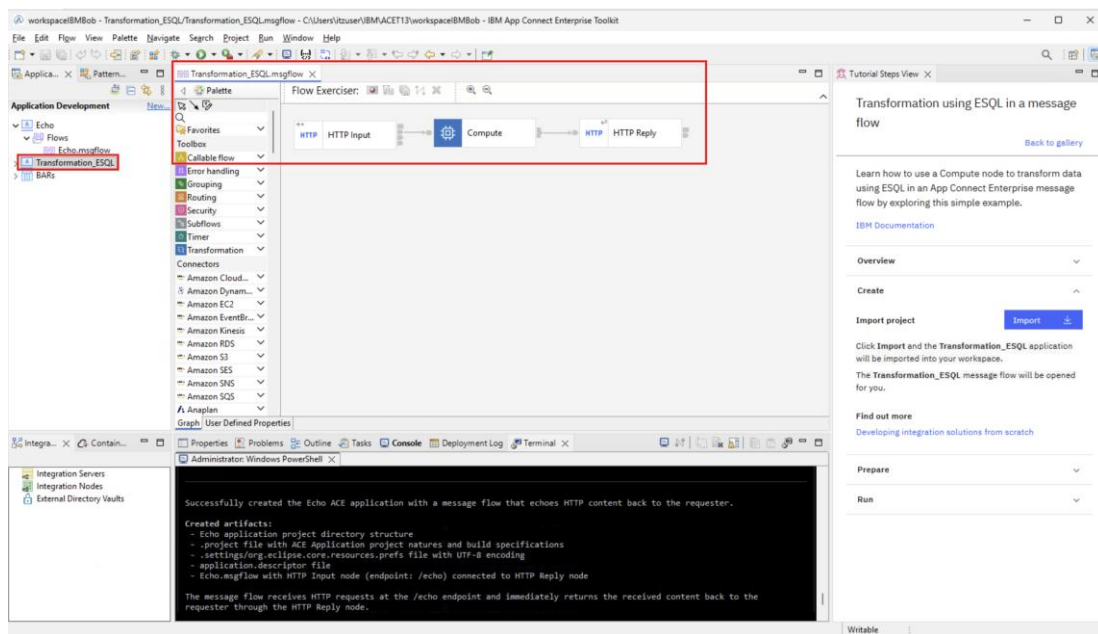
40. Scroll down the list and look for the tutorial named **Transformation using ESQL in a message flow**, and click the **Start** button:



41. The tutorial instructions will appear on the right side of the Toolkit. Click the **Import** button:



42. After a few seconds the import will complete and you will see an application called **Transformation_ESQL** and a message flow inside it of the same name:



43. Next, go back to the IBM Bob console which should still be open and visible at the bottom of the Toolkit. This time, we will type the following prompt which will ask IBM Bob to analyze the message flow which we just imported into the workspace:

Explain the purpose of the message flow called Transformation_ESQL including a summary of the Compute node's ESQL logic.

```
> Explain the purpose of the message flow called Transformation_ESQL including a summary of the Compute node's ESQL logic.
```

44. The IBM Bob Shell describes the analysis. IBM Bob examines the message flow, locates the Compute node within it, and finds the name of the ESQL file with which it is associated:

```
<thinking>Need to find and read the ESQL file
The message flow shows a Compute node that references "esql://routine/#Transformation_Compute.Main". I need to find the
corresponding ESQL file to understand the transformation logic.
</thinking>

FindFiles '**/Transformation_Compute.esql'

Found 1 matching file(s)

<thinking>Read ESQL file to understand Compute node logic
I found the message flow and now need to read the ESQL file to understand what the Compute node does.
</thinking>

ReadFile Tra\...\Transformation_Compute.esql

<thinking>Analyzing Transformation ESQL message flow
I've read both the message flow and the ESQL file. Now I can provide a comprehensive explanation of the purpose and logic.
</thinking>
```

45. The final summary provided by IBM Bob discusses the input and output structures, summarizes the key transformations that are applied, including references to specific data fields, and the logic that is being executed:

```
Administrator: Windows PowerShell X

The Compute node (Transformation_Compute.esql) performs the following transformation:

1. Input Structure: Processes SaleEnvelope/SaleList/Invoice elements containing customer invoices with items
2. Output Structure: Creates SaleEnvelope/SaleList/Statement elements with transformed data
3. Key Transformations:
  - Combines customer initials from two Initial fields into a single Initials field
  - Renames Surname to Name
  - Preserves Balance field
  - Transforms each Item into an Article with:
    - Description → Desc
    - Price multiplied by 1.6 → Cost (applies 60% markup)
    - Quantity → Qty
  - Calculates total amount for all articles (Cost x Quantity)
  - Adds statement attributes: Type="Monthly" and Style="Full"
  - Preserves currency information from the invoice
4. Processing Logic:
  - Uses REFERENCE variables for efficient navigation
  - Iterates through multiple invoices using WHILE loops
  - Nested loop processes all items within each invoice
  - Accumulates totals for each statement
  - Uses proper XMLNSC domain handling for XML manipulation

The transformation essentially converts detailed invoice records into monthly customer statements with price adjustments and
reformatted structure.

You can ask follow up questions or use the /clear command to start a new task
```

8. Use IBM Bob to create a simple file flow and then test

46. Go back to the IBM Bob console which should still be open and visible at the bottom of the Toolkit. This time, we will type the following prompt which will ask IBM Bob to create a new message flow:

Create me an ACE message flow inside the project called Example which reads in a file called input.xml from within the project, and transforms it to a JSON message using a Compute node and writes the content to an output file in the same directory to be called output.json

```
> Create me an ACE message flow inside the project called Example which reads in a file called input.xml from within the project, and transforms it to a JSON message using a Compute node and writes the content to an output file in the same directory to be called output.json
```

47. IBM Bob spots that the project already exists, so this time we don't need to wait for IBM Bob to create the project or the metadata files that we've already seen in the first example in the Lab guide

```
<thinking>**Creating ACE message flow with file input/output**
I need to create a message flow in the Example project that:
1. Reads input.xml file
2. Transforms XML to JSON using a Compute node
3. Writes output to output.json file

This is a multi-step task, so I should create a todo list first.
</thinking>
```

To-do List Updating 7 item(s)

```
1
2 [ ] Create FileToJson.msgflow in Example project
3 [ ] Add FileInput node to read input.xml
4 [ ] Add Compute node for XML to JSON transformation
5 [ ] Write ESQl transformation code
6 [ ] Add FileOutput node to write output.json
7 [ ] Configure node properties and connections
8 [ ] Test the message flow
```

Apply this change?

```
> 1. Yes, allow once
2. Yes, allow always
3. Modify with external editor: none (select one with /editor command)
4. No, suggest changes (esc)
```

48. IBM Bob then starts creating the required message flow:

```
WriteFile Writing to Example\FileToJson.msgflow

... first 16 lines hidden ...
16 <translation xmi:type="utility:ConstantString" string="File Output"/>
17 </nodes>
18 <connections xmi:type="eflow:FCMConnection" xmi:id="FCMConnection_1" targetNode="FCMComposite_1_2" sourceNode=
"FCMComposite_1_1" sourceTerminalName="OutTerminal.out" targetTerminalName="InTerminal.in"/>
19 <connections xmi:type="eflow:FCMConnection" xmi:id="FCMConnection_2" targetNode="FCMComposite_1_3" sourceNode=
"FCMComposite_1_2" sourceTerminalName="OutTerminal.out" targetTerminalName="InTerminal.in"/>
20 </composition>
21 <propertyOrganizer/>
22 <stickyBoard/>
23 </eClassifiers>
24 </ecore:EPackage>

Apply this change?

> 1. Yes, allow once
2. Yes, allow always
3. Modify with external editor: none (select one with /editor command)
4. No, suggest changes (esc)
```

49. IBM Bob recognises the need for a Compute node and associated (relatively trivial) ESQL:

```
Administrator: Windows PowerShell X
<thinking>**Create ESQL transformation code**
I've created the message flow file with FileInput, Compute, and FileOutput nodes. Now I need to create the ESQL code that will
transform the XML to JSON. The Compute node references "esql://routine/#FileToJSON_Compute.Main", so I need to create a file call
ed
FileToJSON_Compute.esql with a Main procedure.
</thinking>

WriteFile Writing to Example\FileToJSON_Compute.esql

1 CREATE COMPUTE MODULE FileToJSON_Compute
2   CREATE FUNCTION Main() RETURNS BOOLEAN
3   BEGIN
4     -- Create output message as JSON
5     SET OutputRoot.Properties = InputRoot.Properties;
6     SET OutputRoot.JSON.Data = InputRoot.XMLNSC.SaleEnvelope;
7
8     RETURN TRUE;
9   END;
10 END MODULE;

Apply this change?
> 1. Yes, allow once
2. Yes, allow always
3. Modify with external editor: none (select one with /editor command)
4. No, suggest changes (esc)

Waiting for user confirmation...
```

50. IBM Bob then returns to assess progress against the initial To-do list:

```
Administrator: Windows PowerShell X
<thinking>**Update todo list with completed items**
I've successfully created the FileToJSON.msgflow with all three nodes (FileInput, Compute, FileOutput) configured and connected,
and also created the ESQL transformation code. Now I need to update the todo list to reflect completion.
</thinking>

To-do List Updating 7 item(s)

... first 4 lines hidden ...
5 - [ ] Add FileOutput node to write output.json
6 - [ ] Configure node properties and connections
7 - [ ] Test the message flow

1 +
2 + [x] Create FileToJSON.msgflow in Example project
3 + [x] Add FileInput node to read input.xml
4 + [x] Add Compute node for XML to JSON transformation
5 + [x] Write ESQL transformation code
6 + [x] Add FileOutput node to write output.json
7 + [x] Configure node properties and connections
8 + [ ] Test the message flow (user needs to deploy and test in ACE Toolkit)

Apply this change?
> 1. Yes, allow once
2. Yes, allow always
3. Modify with external editor: none (select one with /editor command)
4. No, suggest changes (esc)

Press ctrl-s to show more lines
Waiting for user confirmation...
```

51. Finally IBM Bob summarizes the tasks that have been completed:

```
Task Completion Successfully completed | Cost: 0.22

Successfully created ACE message flow in the Example project:

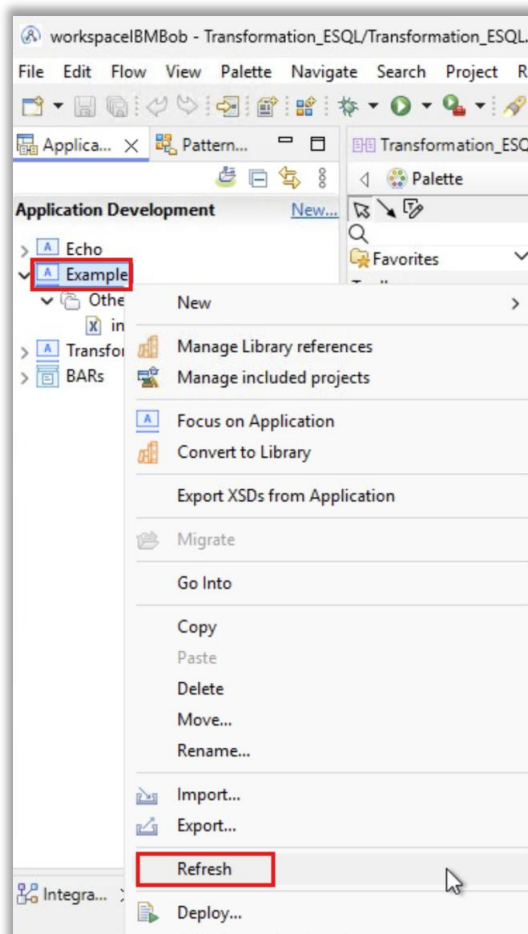
**Created Files:**
1. **FileToJSON.msgflow** - Message flow with three nodes:
  - File Input: Reads input.xml from the Example project directory
  - Compute: Transforms XML to JSON using ESQL
  - File Output: Writes output.json to the same directory
2. **FileToJSON_Compute.esql** - ESQL transformation module that converts the XML SaleEnvelope structure to JSON format

**Next Steps:**
To use the message flow:
1. Open the Example project in IBM App Connect Enterprise Toolkit
2. Deploy the FileToJSON message flow to your integration server
3. The flow will automatically read input.xml and create output.json with the transformed JSON content

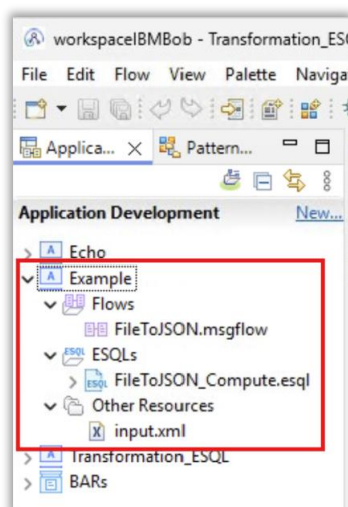
The transformation preserves the complete XML structure including the Header, SaleList with Invoices, Items, and Trailer sections.

You can ask follow up questions or use the /clear command to start a new task
```

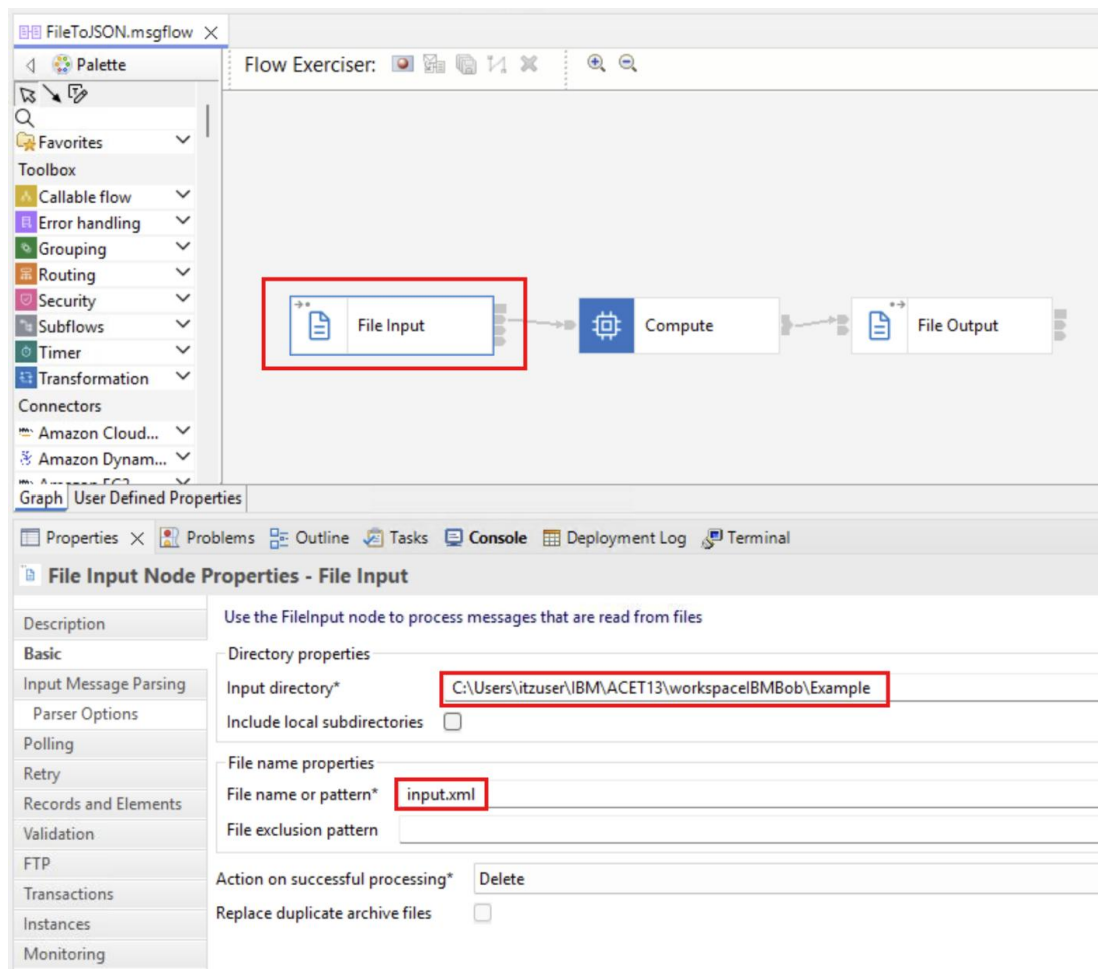
52. With this example we will go a step further and deploy and test the example that Bob has created. Like before, we want to examine and use the artifacts that IBM Bob generated, but because this time the Eclipse project already existed before the actions were taken, instead of importing the generated Eclipse project, all we need to do is to right-click the application project named **Example** and choose the **Refresh** option:



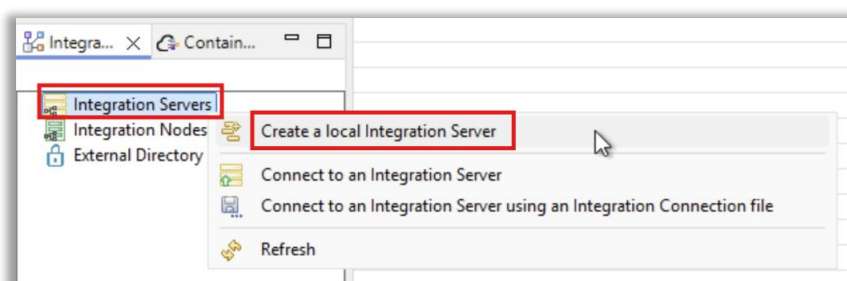
53. The contents of the Example application project will be refreshed and you should now see the message flow and associated ESQL file inside the project as shown below:



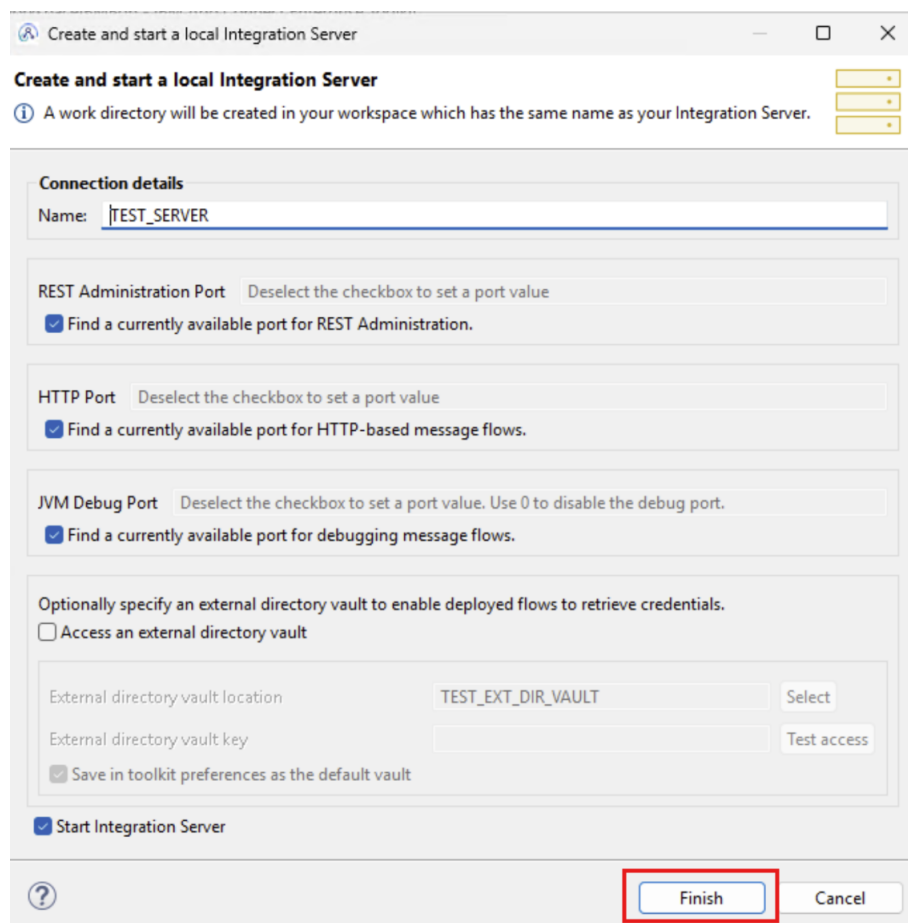
54. Open the generated message flow (in our example this has been named **FileToJSON.msgflow**, but the name chosen by IBM Bob in your case may have been different) by double clicking it. Inspect the properties of the nodes. In particular pay close attention to the FileInput node and its properties for **Input directory** and **File name or pattern**:



55. Right click on the Integration Servers level of the tree in the Integration Explorer view in the bottom left part of the Toolkit and choose the menu option to **Create a local Integration Server**



56. Stick with all of the presented defaults (for this simple example we don't even need to create an external directory vault as we have no need to store any credentials) and just click the **Finish** button:



Create and start a local Integration Server

Create and start a local Integration Server

A work directory will be created in your workspace which has the same name as your Integration Server.

Connection details

Name:

REST Administration Port Deselect the checkbox to set a port value
☒ Find a currently available port for REST Administration.

HTTP Port Deselect the checkbox to set a port value
☒ Find a currently available port for HTTP-based message flows.

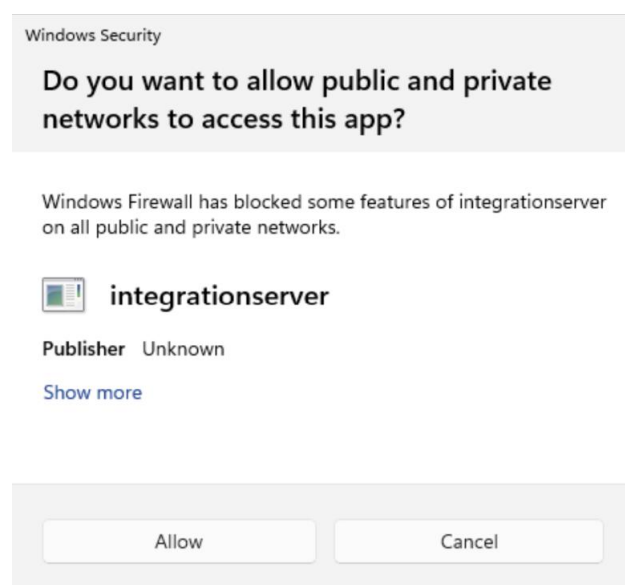
JVM Debug Port Deselect the checkbox to set a port value. Use 0 to disable the debug port.
☒ Find a currently available port for debugging message flows.

Optional specify an external directory vault to enable deployed flows to retrieve credentials.
☐ Access an external directory vault

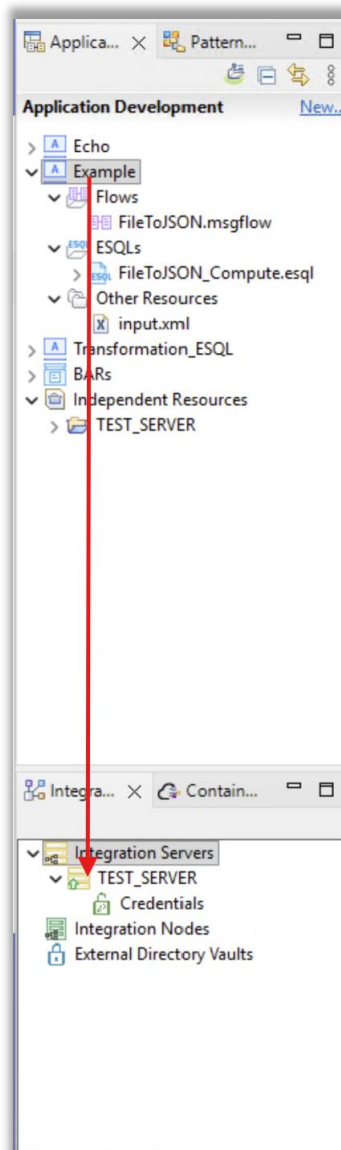
External directory vault location
External directory vault key
☒ Save in toolkit preferences as the default vault

☒ Start Integration Server

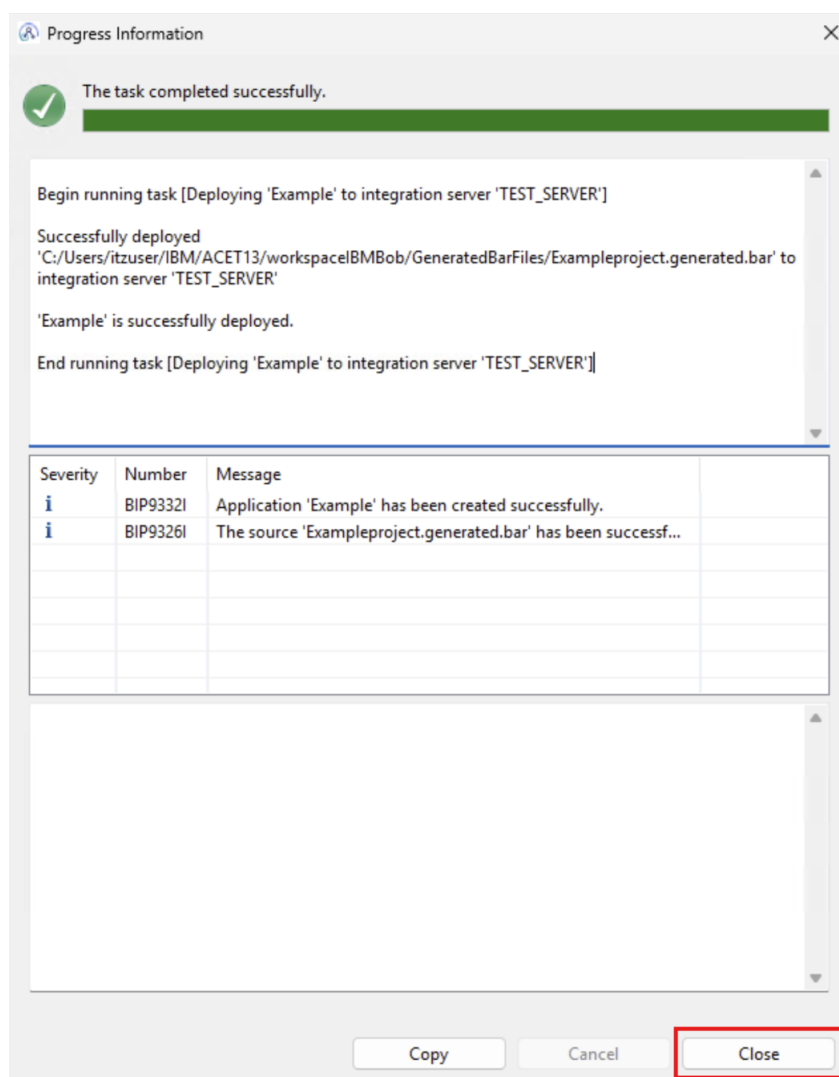
The Integration Server will start. If you receive the following warning you should click **Allow**:



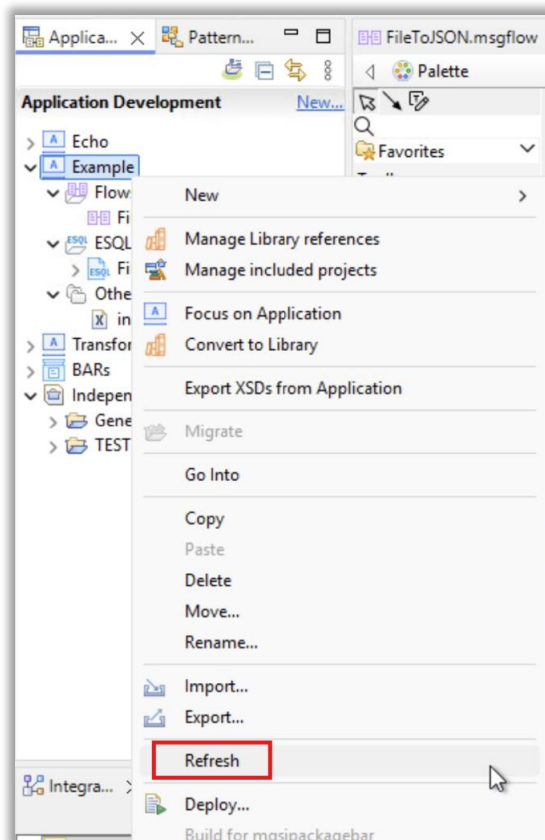
57. Now do a drag and drop deploy of the project into the integration server (**TEST_SERVER**) which we just created:



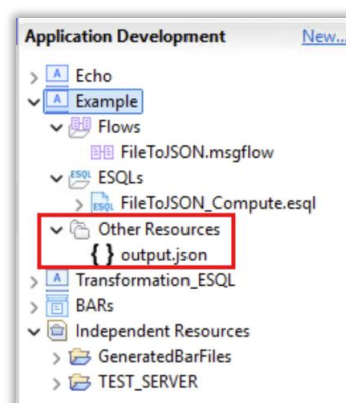
58. You should receive a successful confirmation message for the deploy. Dismiss the message by clicking the **Close** button:



59. As soon as the application is deployed, we would expect the message flow to process the input file (input.xml) from inside the local Eclipse project. Right click the project in the Application Development view and choose the menu option to **Refresh**:



60. Once refreshed, you should observe that the **input.xml** file has been processed so is no longer there, and in its place **output.json** file will be there:



- | | |
|-----|---|
| 61. | <p>In summary we have demonstrated a few simple use cases with IBM Bob in the ACE Toolkit. You should have sufficient IBM Bob coins left in your trial to try some other prompts.</p> <p>Over time we expect to improve the ace-bob skill to extend the reach of what is possible with this technology so if you have suggestions for which improvements should be made next, then IBM would love to hear your views.</p> |
|-----|---|

END OF LAB GUIDE